

Обучение по демонстрациям и трансформеры

Алексей Скрынник

ПЛАН

- 01 Поведенческое клонирование
- 02 Обучение с экспертом и симулятором
- 03 Offline RL
- 04 Обучение по экспертному поведению в симуляторе
- 05 Обучение по демонстрациям в среде
- 06 Обучение по поведению в среде

01

Поведенческое клонирование

Зачем учиться по демонстрациям?

Классическая проблема в обучении с подкреплением: агент должен сам найти сильную стратегию методом проб и ошибок.

Зачем учиться по демонстрациям?

Классическая проблема в обучении с подкреплением: агент должен сам найти сильную стратегию методом проб и ошибок.

Но это может быть дорого или небезопасно:

- **Робототехника:** случайное исследование может повредить оборудование
- **Автономное вождение:** опасное исследование недопустимо
- **Игры:** разреженные награды могут требовать миллионы попыток

Зачем учиться по демонстрациям?

Классическая проблема в обучении с подкреплением: агент должен сам найти сильную стратегию методом проб и ошибок.

Но это может быть дорого или небезопасно:

- **Робототехника:** случайное исследование может повредить оборудование
- **Автономное вождение:** опасное исследование недопустимо
- **Игры:** разреженные награды могут требовать миллионы попыток

Идея обучения по демонстрациям: начать с поведения, которое уже работает.

$$\mathcal{D}_E = \{(o_i, a_i)\}_{i=1}^N$$

Зачем учиться по демонстрациям?

Классическая проблема в обучении с подкреплением: агент должен сам найти сильную стратегию методом проб и ошибок.

Но это может быть дорого или небезопасно:

- **Робототехника:** случайное исследование может повредить оборудование
- **Автономное вождение:** опасное исследование недопустимо
- **Игры:** разреженные награды могут требовать миллионы попыток

Идея обучения по демонстрациям: начать с поведения, которое уже работает.

$$\mathcal{D}_E = \{(o_i, a_i)\}_{i=1}^N$$

Обучаем стратегию, которая сопоставляет наблюдения с действиями, похожими на экспертные:

$$\pi_{\theta}(a \mid o) \approx \pi_E(a \mid o)$$

Что доступно в базовом ВС?

Есть только записанное поведение:

$$\mathcal{D}_E = \{(o_i, a_i)\}_{i=1}^N$$

- не запускаем стратегию в среде;
- не запрашиваем новые действия эксперта;
- не используем функцию вознаграждения.



Поведенческое клонирование = обучение с учителем

Вход: экспертные пары наблюдение-действие $\mathcal{D}_E = \{(o_i, a_i)\}_{i=1}^N$

Цель обучения — предсказывать действия эксперта:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N -\log \pi_{\theta}(a_i \mid o_i)$$

Поведенческое клонирование = обучение с учителем

Вход: экспертные пары наблюдение-действие $\mathcal{D}_E = \{(o_i, a_i)\}_{i=1}^N$

Цель обучения — предсказывать действия эксперта:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N -\log \pi_{\theta}(a_i \mid o_i)$$

Дискретные действия: категориальная стратегия, кросс-энтропийная ошибка

$$\pi_{\theta}(a \mid o) = \text{Categorical}(f_{\theta}(o))$$

Поведенческое клонирование = обучение с учителем

Вход: экспертные пары наблюдение-действие $\mathcal{D}_E = \{(o_i, a_i)\}_{i=1}^N$

Цель обучения — предсказывать действия эксперта:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N -\log \pi_{\theta}(a_i \mid o_i)$$

Дискретные действия: категориальная стратегия, кросс-энтропийная ошибка

$$\pi_{\theta}(a \mid o) = \text{Categorical}(f_{\theta}(o))$$

Непрерывные действия: гауссовская стратегия

$$\pi_{\theta}(a \mid o) = \mathcal{N}(a; \mu_{\theta}(o), \Sigma_{\theta}(o))$$

Поведенческое клонирование = обучение с учителем

Вход: экспертные пары наблюдение-действие $\mathcal{D}_E = \{(o_i, a_i)\}_{i=1}^N$

Цель обучения — предсказывать действия эксперта:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N -\log \pi_{\theta}(a_i \mid o_i)$$

Дискретные действия: категориальная стратегия, кросс-энтропийная ошибка

$$\pi_{\theta}(a \mid o) = \text{Categorical}(f_{\theta}(o))$$

Непрерывные действия: гауссовская стратегия

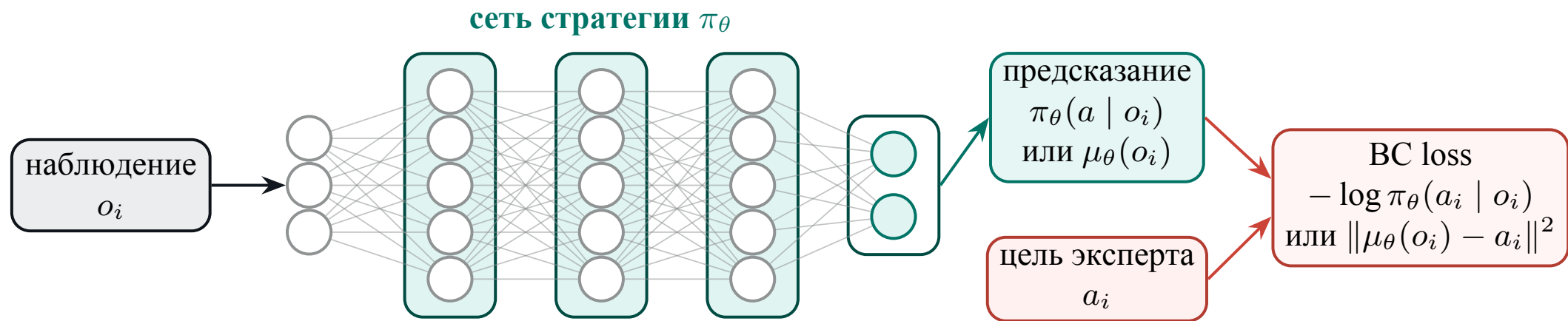
$$\pi_{\theta}(a \mid o) = \mathcal{N}(a; \mu_{\theta}(o), \Sigma_{\theta}(o))$$

Частный случай с фиксированной ковариацией:

$$-\log \pi_{\theta}(a_i \mid o_i) \propto \|\mu_{\theta}(o_i) - a_i\|_2^2$$

Ключевое преимущество: не нужны функция вознаграждения, симулятор и online-исследование.

Что именно мы обучаем?



Что на самом деле клонирует ВС?

Экспертные данные

$$\tau = (o_1, a_1, o_2, a_2, \dots)$$

- наблюдения: состояния, изображения, текст, карты
- действия: управляющие сигналы, нажатия кнопок, токены

Что на самом деле клонирует ВС?

Экспертные данные

$$\tau = (o_1, a_1, o_2, a_2, \dots)$$

- наблюдения: состояния, изображения, текст, карты
- действия: управляющие сигналы, нажатия кнопок, токены

Модель стратегии

$$\pi_{\theta}(a_t \mid o_{\leq t}, a_{< t})$$

- MLP для низкоразмерных состояний
- CNN для визуальных наблюдений
- RNN / Transformer для поведения, зависящего от истории

Что на самом деле клонирует ВС?

Экспертные данные

$$\tau = (o_1, a_1, o_2, a_2, \dots)$$

- наблюдения: состояния, изображения, текст, карты
- действия: управляющие сигналы, нажатия кнопок, токены

Модель стратегии

$$\pi_{\theta}(a_t \mid o_{\leq t}, a_{< t})$$

- MLP для низкоразмерных состояний
- CNN для визуальных наблюдений
- RNN / Transformer для поведения, зависящего от истории

ВС не учится понимать, **почему** эксперт действует именно так. Он учит отображение вход-выход, присутствующее в демонстрациях.

Сильные примеры поведенческого клонирования

Робототехника

- данные телеуправления
- навыки манипуляции
- зрительно-моторные стратегии



Сильные примеры поведенческого клонирования

Робототехника

- данные телеуправления
- навыки манипуляции
- зрительно-моторные стратегии



Автономное вождение

- кадры с камеры → угол поворота руля
- удержание полосы
- управление с учетом маршрута

Сильные примеры поведенческого клонирования

Робототехника

- данные телеуправления
- навыки манипуляции
- зрительно-моторные стратегии



Автономное вождение

- кадры с камеры → угол поворота руля
- удержание полосы
- управление с учетом маршрута

Игры и симуляторы

- записи игр людей
- экспертные боты
- траектории решенных планировщиком задач

Сильные примеры поведенческого клонирования

Робототехника

- данные телеуправления
- навыки манипуляции
- зрительно-моторные стратегии



Автономное вождение

- кадры с камеры → угол поворота руля
- удержание полосы
- управление с учетом маршрута

Игры и симуляторы

- записи игр людей
- экспертные боты
- траектории решенных планировщиком задач

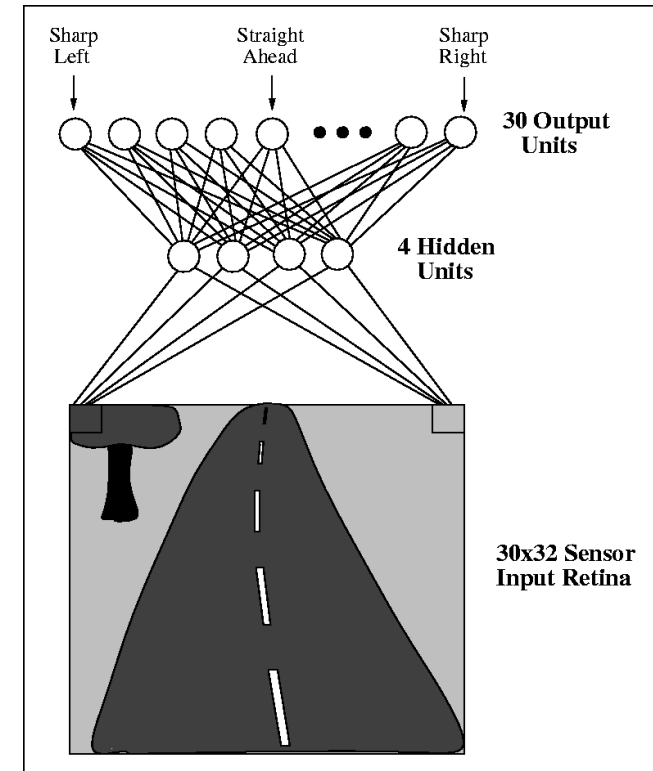
Почему ВС может быть неожиданно сильным

Если экспертных данных много, они разнообразны и хорошо совпадают с состояниями на тесте, обучение с учителем может дать сильные стратегии еще до дообучения через RL.

Пример: ALVINN

ALVINN

- ранняя нейросетевая система вождения
- изображения дороги → поворот руля
- классический пример прямой имитации



Пример: AlphaStar

Предобучение AlphaStar

- сначала обучался на играх людей
- имитация дает сильную начальную стратегию
- RL и self-play улучшают ее позже



Пример: VPT для Minecraft

VPT для Minecraft

- учится по видео прохождений людей
- восстанавливает действия через inverse dynamics model
- ВС дает стратегию, которую затем дообучают RL



Главная слабость: ковариационный сдвиг

Пусть d^π обозначает распределение наблюдений, получаемые при исполнении стратегии π .

Обучение: экспертные состояния $o \sim d^{\pi_E}$

Главная слабость: ковариационный сдвиг

Пусть d^π обозначает распределение наблюдений, получаемые при исполнении стратегии π .

Обучение: экспертные состояния $o \sim d^{\pi_E}$

Исполнение: состояния, порожденные учеником, $o \sim d^{\pi_\theta}$

Главная слабость: ковариационный сдвиг

Пусть d^π обозначает распределение наблюдений, получаемые при исполнении стратегии π .

Обучение: экспертные состояния $o \sim d^{\pi_E}$

Исполнение: состояния, порожденные учеником, $o \sim d^{\pi_\theta}$

Даже небольшая ошибка может привести агента в состояния, которых нет в датасете:

$$d^{\pi_\theta} \neq d^{\pi_E}$$

Главная слабость: ковариационный сдвиг

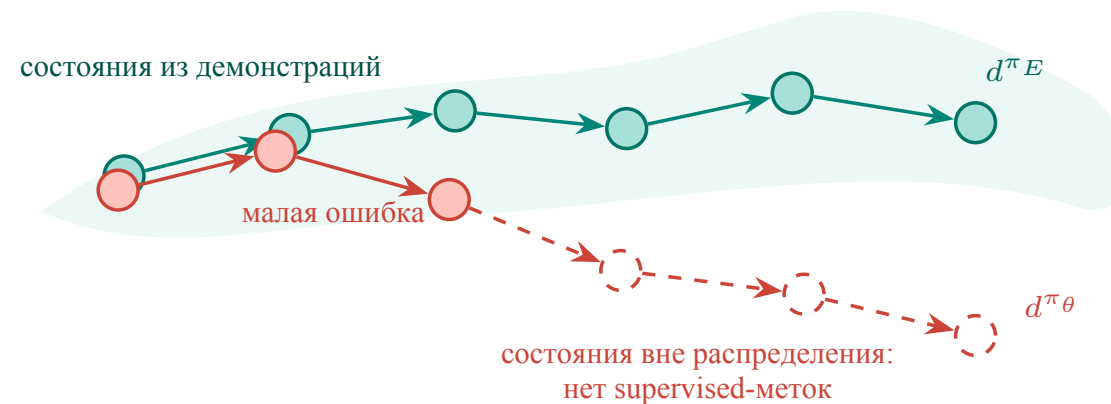
Пусть d^π обозначает распределение наблюдений, получаемые при исполнении стратегии π .

Обучение: экспертные состояния $o \sim d^{\pi_E}$

Исполнение: состояния, порожденные учеником, $o \sim d^{\pi_\theta}$

Даже небольшая ошибка может привести агента в состояния, которых нет в датасете:

$$d^{\pi_\theta} \neq d^{\pi_E}$$



Почему малые ошибки становятся большими

Пусть клонированная стратегия имеет вероятность ошибки ϵ на каждом шаге в экспертных состояниях.

Почему малые ошибки становятся большими

Пусть клонированная стратегия имеет вероятность ошибки ϵ на каждом шаге в экспертных состояниях.

На горизонте T вероятность хотя бы одной ошибки примерно равна:

$$1 - (1 - \epsilon)^T \approx T\epsilon$$

Почему малые ошибки становятся большими

Пусть клонированная стратегия имеет вероятность ошибки ϵ на каждом шаге в экспертных состояниях.

На горизонте T вероятность хотя бы одной ошибки примерно равна:

$$1 - (1 - \epsilon)^T \approx T\epsilon$$

После ошибки агент может попасть в состояния, где у supervised-модели не было обучающего сигнала.

Почему малые ошибки становятся большими

Пусть клонированная стратегия имеет вероятность ошибки ϵ на каждом шаге в экспертных состояниях.

На горизонте T вероятность хотя бы одной ошибки примерно равна:

$$1 - (1 - \epsilon)^T \approx T\epsilon$$

После ошибки агент может попасть в состояния, где у supervised-модели не было обучающего сигнала.

Практический вывод

ВС наиболее силен, когда демонстрации покрывают восстановительное поведение, задача имеет короткий горизонт или стратегия позже улучшается через взаимодействие.

Как усилить ВС

1. Более разнообразные демонстрации

- включать разные старты, цели, разметки среды и операторов

Как усилить ВС

1. Более разнообразные демонстрации

- включать разные старты, цели, разметки среды и операторов

2. Данные восстановления

- показать стратегии, как возвращаться из почти провальных состояний

Как усилить ВС

1. Более разнообразные демонстрации

- включать разные старты, цели, разметки среды и операторов

2. Данные восстановления

- показать стратегии, как возвращаться из почти провальных состояний

3. История и память

- использовать RNN или Transformers, когда текущее наблюдение неоднозначно

Как усилить ВС

1. Более разнообразные демонстрации

- включать разные старты, цели, разметки среды и операторов

2. Данные восстановления

- показать стратегии, как возвращаться из почти провальных состояний

3. История и память

- использовать RNN или Transformers, когда текущее наблюдение неоднозначно

4. Лучшие распределения действий

- mixture density, дискретизация или диффузионные стратегии для мультимодальных действий

Как усилить ВС

1. Более разнообразные демонстрации

- включать разные старты, цели, разметки среды и операторов

2. Данные восстановления

- показать стратегии, как возвращаться из почти провальных состояний

3. История и память

- использовать RNN или Transformers, когда текущее наблюдение неоднозначно

4. Лучшие распределения действий

- mixture density, дискретизация или диффузионные стратегии для мультимодальных действий

5. Интерактивная коррекция

- DAgger размечает состояния, посещенные обученной стратегией

Gato: масштабный пример ВС

Generalist agent

- одна Transformer-модель
- много доменов: игры, робототехника, текст, изображения
- разные действия: дискретные токены и непрерывное управление

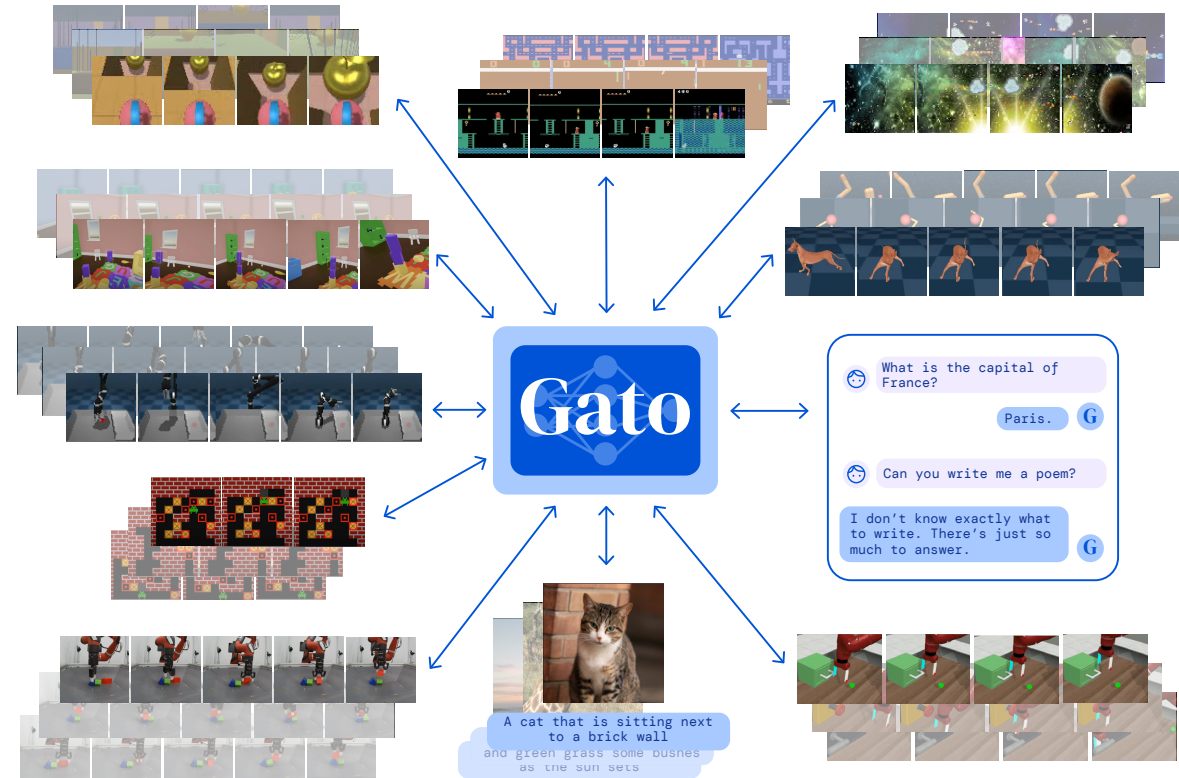
Gato: масштабный пример ВС

Generalist agent

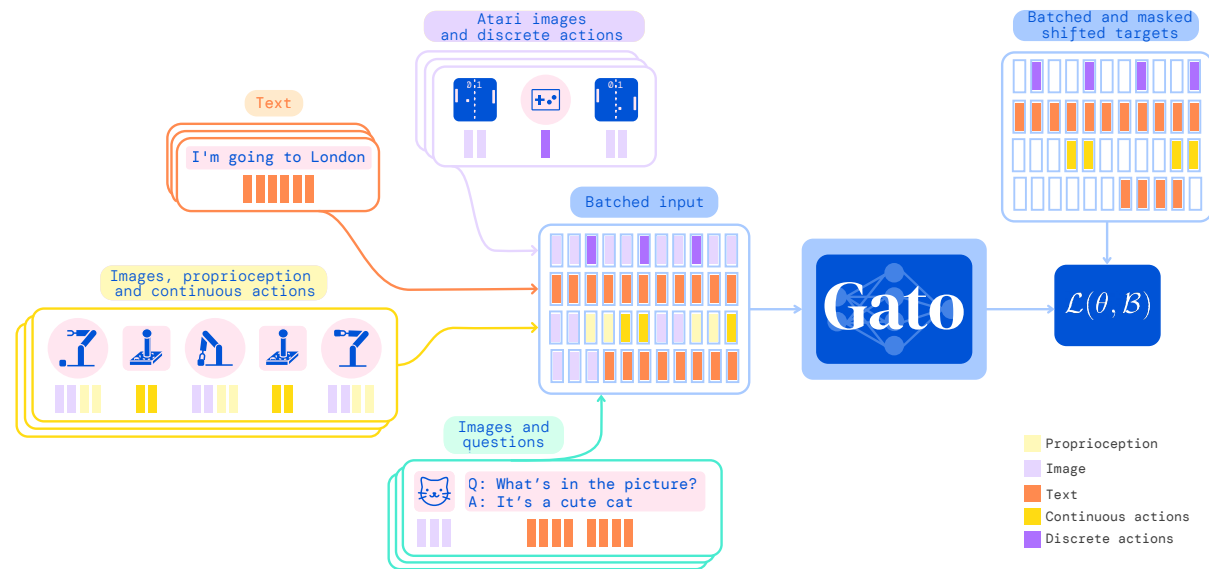
- одна Transformer-модель
- много доменов: игры, робототехника, текст, изображения
- разные действия: дискретные токены и непрерывное управление

Идея

Свести разные демонстрации к
одному формату и обучать
следующему токену или действию.



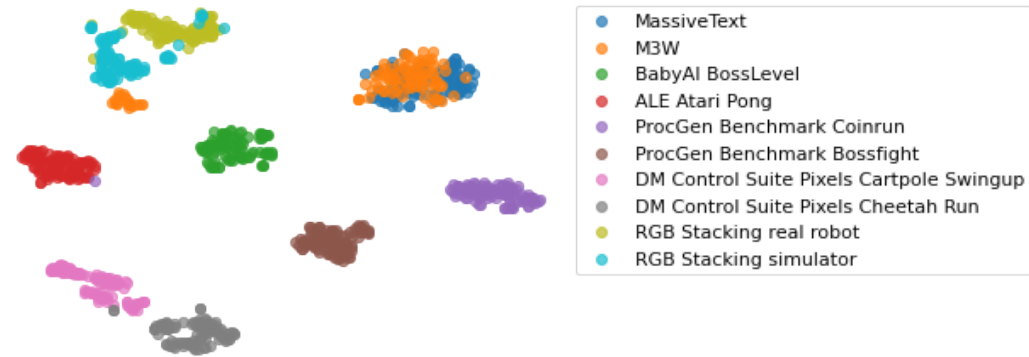
Gato: демонстрации как последовательности



$$\max_{\theta} \mathbb{E}_{x_{1:T} \sim \mathcal{D}_{\text{Gato}}} \sum_{t=1}^T \log p_{\theta}(x_{t+1} \mid x_{\leq t})$$

Для действий: $\log \pi_{\theta}(a_t \mid \text{контекст})$ — обычный ВС.

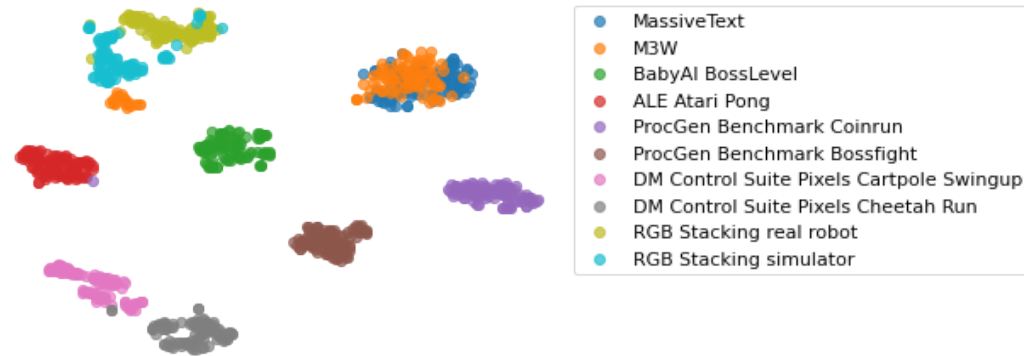
Gato: один loss для разных доменов



Что объединяется

- текстовые токены
- патчи изображений
- proprioception
- действия в разных средах

Gato: один loss для разных доменов



Что объединяется

- текстовые токены
- патчи изображений
- proprioception
- действия в разных средах

Что остается одинаковым

- causal Transformer
- маскированная supervised-цель
- обучение на зафиксированных демонстрациях

Поведенческое клонирование: выводы

Сильные стороны

- простая supervised-цель
- не нужна настройка награды
- нет online-исследования
- естественно масштабируется с объемом данных

Поведенческое клонирование: выводы

Сильные стороны

- простая supervised-цель
- не нужна настройка награды
- нет online-исследования
- естественно масштабируется с объемом данных

Слабые стороны

- сдвиг распределения
- накопление ошибок
- само по себе не превосходит продемонстрированное поведение
- чувствительно к качеству датасета

02

Обучение с экспертом и симулятором

Зачем добавлять симулятор?

Клонирование поведения учится только на состояниях из экспертных демонстраций:

$$O \sim d^{\pi_E}.$$

Зачем добавлять симулятор?

Клонирование поведения учится только на состояниях из экспертных демонстраций:

$$o \sim d^{\pi_E}.$$

Но при запуске ученик попадает уже в другие состояния:

$$o \sim d^{\pi_\theta}.$$

Зачем добавлять симулятор?

Клонирование поведения учится только на состояниях из экспертных демонстраций:

$$o \sim d^{\pi_E}.$$

Но при запуске ученик попадает уже в другие состояния:

$$o \sim d^{\pi_\theta}.$$

Идея: запустить текущую стратегию, увидеть, в какие состояния она попадает, и спросить эксперта, как действовать в этих состояниях.

Зачем добавлять симулятор?

Клонирование поведения учится только на состояниях из экспертных демонстраций:

$$o \sim d^{\pi_E}.$$

Но при запуске ученик попадает уже в другие состояния:

$$o \sim d^{\pi_\theta}.$$

Идея: запустить текущую стратегию, увидеть, в какие состояния она попадает, и спросить эксперта, как действовать в этих состояниях.

Так мы добавляем в датасет состояния, возникающие из-за ошибок ученика, и получаем для них правильные действия эксперта.

Что доступно: эксперт + симулятор

В этом разделе предполагаем:

- есть эксперт, который может подсказать действие в данном состоянии;
- есть симулятор, где можно запускать ученика;
- новые данные собираются во время запусков в симуляторе.

Что доступно: эксперт + симулятор

В этом разделе предполагаем:

- есть эксперт, который может подсказать действие в данном состоянии;
- есть симулятор, где можно запускать ученика;
- новые данные собираются во время запусков в симуляторе.

эксперт

симулятор

Главный вопрос: **в каких состояниях спрашивать эксперта**, чтобы исправить ошибки ученика?

Алгоритм DAgger

Вход: демонстрации $\mathcal{D}_E$, симулятор, экспертная стратегия π_E

Алгоритм DAgger

Вход: демонстрации $\mathcal{D}_E$, симулятор, экспертная стратегия π_E

1: Инициализировать $\mathcal{D}_0 \leftarrow \mathcal{D}_E$

Алгоритм DAgger

Вход: демонстрации $\mathcal{D}_E$, симулятор, экспертная стратегия π_E

- 1: Инициализировать $\mathcal{D}_0 \leftarrow \mathcal{D}_E$
- 2: Обучить начальную стратегию π_1 на $\mathcal{D}_0$

Алгоритм DAgger

Вход: демонстрации $\mathcal{D}_E$, симулятор, экспертная стратегия π_E

- 1: Инициализировать $\mathcal{D}_0 \leftarrow \mathcal{D}_E$
- 2: Обучить начальную стратегию π_1 на $\mathcal{D}_0$
- 3: **for** $k = 1, \dots, K$ **do**
- 4: Запустить текущую стратегию π_k в симуляторе

Алгоритм DAgger

Вход: демонстрации $\mathcal{D}_E$, симулятор, экспертная стратегия π_E

- 1: Инициализировать $\mathcal{D}_0 \leftarrow \mathcal{D}_E$
- 2: Обучить начальную стратегию π_1 на $\mathcal{D}_0$
- 3: **for** $k = 1, \dots, K$ **do**
- 4: Запустить текущую стратегию π_k в симуляторе
- 5: Собрать посещенные состояния $o_1, \dots, o_T$

Алгоритм DAgger

Вход: демонстрации $\mathcal{D}_E$, симулятор, экспертная стратегия π_E

- 1: Инициализировать $\mathcal{D}_0 \leftarrow \mathcal{D}_E$
- 2: Обучить начальную стратегию π_1 на $\mathcal{D}_0$
- 3: **for** $k = 1, \dots, K$ **do**
- 4: Запустить текущую стратегию π_k в симуляторе
- 5: Собрать посещенные состояния $o_1, \dots, o_T$
- 6: Запросить у эксперта действия $a_t^E \sim \pi_E(\cdot \mid o_t)$

Алгоритм DAgger

Вход: демонстрации $\mathcal{D}_E$, симулятор, экспертная стратегия π_E

- 1: Инициализировать $\mathcal{D}_0 \leftarrow \mathcal{D}_E$
- 2: Обучить начальную стратегию π_1 на $\mathcal{D}_0$
- 3: **for** $k = 1, \dots, K$ **do**
- 4: Запустить текущую стратегию π_k в симуляторе
- 5: Собрать посещенные состояния $o_1, \dots, o_T$
- 6: Запросить у эксперта действия $a_t^E \sim \pi_E(\cdot \mid o_t)$
- 7: Агрегировать данные $\mathcal{D}_k \leftarrow \mathcal{D}_{k-1} \cup \{(o_t, a_t^E)\}_{t=1}^T$

Алгоритм DAgger

Вход: демонстрации $\mathcal{D}_E$, симулятор, экспертная стратегия π_E

- 1: Инициализировать $\mathcal{D}_0 \leftarrow \mathcal{D}_E$
- 2: Обучить начальную стратегию π_1 на $\mathcal{D}_0$
- 3: **for** $k = 1, \dots, K$ **do**
- 4: Запустить текущую стратегию π_k в симуляторе
- 5: Собрать посещенные состояния $o_1, \dots, o_T$
- 6: Запросить у эксперта действия $a_t^E \sim \pi_E(\cdot \mid o_t)$
- 7: Агрегировать данные $\mathcal{D}_k \leftarrow \mathcal{D}_{k-1} \cup \{(o_t, a_t^E)\}_{t=1}^T$
- 8: Переобучить стратегию π_{k+1} на $\mathcal{D}_k$
- 9: **end for**

Какая цель оптимизируется?

DAgger по-прежнему использует обучение с учителем, но на агрегированном датасете:

$$\theta_{k+1} = \arg \min_{\theta} \frac{1}{|\mathcal{D}_k|} \sum_{(o, a_E) \in \mathcal{D}_k} \ell(\pi_{\theta}(\cdot \mid o), a_E) .$$

Какая цель оптимизируется?

DAgger по-прежнему использует обучение с учителем, но на агрегированном датасете:

$$\theta_{k+1} = \arg \min_{\theta} \frac{1}{|\mathcal{D}_k|} \sum_{(o, a_E) \in \mathcal{D}_k} \ell(\pi_{\theta}(\cdot \mid o), a_E).$$

Типичные функции потерь:

$$\ell = -\log \pi_{\theta}(a_E \mid o) \quad \text{или} \quad \ell = \|\mu_{\theta}(o) - a_E\|_2^2.$$

Какая цель оптимизируется?

DAgger по-прежнему использует обучение с учителем, но на агрегированном датасете:

$$\theta_{k+1} = \arg \min_{\theta} \frac{1}{|\mathcal{D}_k|} \sum_{(o, a_E) \in \mathcal{D}_k} \ell(\pi_{\theta}(\cdot \mid o), a_E).$$

Типичные функции потерь:

$$\ell = -\log \pi_{\theta}(a_E \mid o) \quad \text{или} \quad \ell = \|\mu_{\theta}(o) - a_E\|_2^2.$$

Важное изменение не в функции потерь, а в том, **какие состояния попадают в обучающую выборку.**

DAgger на практике: смешивание стратегий

Ранние стратегии ученика могут быть плохими. Часто запускают смесь стратегий:

$$\pi_{\beta_k} = \beta_k \pi_E + (1 - \beta_k) \pi_k, \quad \beta_k \downarrow 0.$$

DAgger на практике: смешивание стратегий

Ранние стратегии ученика могут быть плохими. Часто запускают смесь стратегий:

$$\pi_{\beta_k} = \beta_k \pi_E + (1 - \beta_k) \pi_k, \quad \beta_k \downarrow 0.$$

- большое β_k : траектории остаются ближе к поведению эксперта
- малое β_k : данные ближе к состояниям, где будет действовать итоговая стратегия
- в обучающую выборку всегда попадают действия эксперта, даже если траекторию ведет ученик

Dagger на практике: смешивание стратегий

Ранние стратегии ученика могут быть плохими. Часто запускают смесь стратегий:

$$\pi_{\beta_k} = \beta_k \pi_E + (1 - \beta_k) \pi_k, \quad \beta_k \downarrow 0.$$

- большое β_k : траектории остаются ближе к поведению эксперта
- малое β_k : данные ближе к состояниям, где будет действовать итоговая стратегия
- в обучающую выборку всегда попадают действия эксперта, даже если траекторию ведет ученик

В робототехнике или вождении это может означать совместное управление, вмешательство системы безопасности или запуски только в симуляции.

Почему DAgger помогает

ВС

- обучается на d^{π_E}
- ошибки уводят стратегию в состояния, которых не было в данных
- нет меток для восстановления

Почему DAgger помогает

BC

- обучается на d^{π_E}
- ошибки уводят стратегию в состояния, которых не было в данных
- нет меток для восстановления

DAgger

- обучается на состояниях ученика
- эксперт показывает, как исправлять ошибки
- уменьшает накопление ошибок

Почему DAgger помогает

BC

- обучается на d^{π_E}
- ошибки уводят стратегию в состояния, которых не было в данных
- нет меток для восстановления

DAgger

- обучается на состояниях ученика
- эксперт показывает, как исправлять ошибки
- уменьшает накопление ошибок

Стратегия учится исправляться после собственных ошибок, а не только повторять идеальные траектории.

DART: устойчивые демонстрации без итеративных запросов

DART = Disturbances for Augmenting Robot Trajectories.

DART: устойчивые демонстрации без итеративных запросов

DART = Disturbances for Augmenting Robot Trajectories.

Вместо многократных запусков ученика и запросов к эксперту добавим шум при сборе демонстраций:

$$a_t = a_t^E + \xi_t.$$

DART: устойчивые демонстрации без итеративных запросов

DART = Disturbances for Augmenting Robot Trajectories.

Вместо многократных запусков ученика и запросов к эксперту добавим шум при сборе демонстраций:

$$a_t = a_t^E + \xi_t.$$

Эксперт естественно исправляет траекторию после возмущений, поэтому в датасете появляются примеры того, как возвращаться к хорошей траектории после небольших отклонений.

DART: устойчивые демонстрации без итеративных запросов

DART = Disturbances for Augmenting Robot Trajectories.

Вместо многократных запусков ученика и запросов к эксперту добавим шум при сборе демонстраций:

$$a_t = a_t^E + \xi_t.$$

Эксперт естественно исправляет траекторию после возмущений, поэтому в датасете появляются примеры того, как возвращаться к хорошей траектории после небольших отклонений.

Компромисс

DART проще, чем DAgger, когда каждый запрос к эксперту дорог, но он помогает только для тех отклонений, которые похожи на шум при сборе данных.

Когда полезен этот подход?

- Есть эксперт или оракул, который может подсказывать действия.

Когда полезен этот подход?

- Есть эксперт или оракул, который может подсказывать действия.
- Можно безопасно запускать ученика в симуляторе.

Когда полезен этот подход?

- Есть эксперт или оракул, который может подсказывать действия.
- Можно безопасно запускать ученика в симуляторе.
- Обычный ВС ошибается, потому что в данных мало примеров восстановления после отклонений.

Когда полезен этот подход?

- Есть эксперт или оракул, который может подсказывать действия.
- Можно безопасно запускать ученика в симуляторе.
- Обычный ВС ошибается, потому что в данных мало примеров восстановления после отклонений.
- RL с нуля слишком дорогой или небезопасный.

Когда полезен этот подход?

- Есть эксперт или оракул, который может подсказывать действия.
- Можно безопасно запускать ученика в симуляторе.
- Обычный ВС ошибается, потому что в данных мало примеров восстановления после отклонений.
- RL с нуля слишком дорогой или небезопасный.

Главное ограничение

Этим методам нужны запуски в среде и новые запросы к эксперту. Если доступен только фиксированный датасет, переходим к offline RL и подходам, которые моделируют последовательности.

03

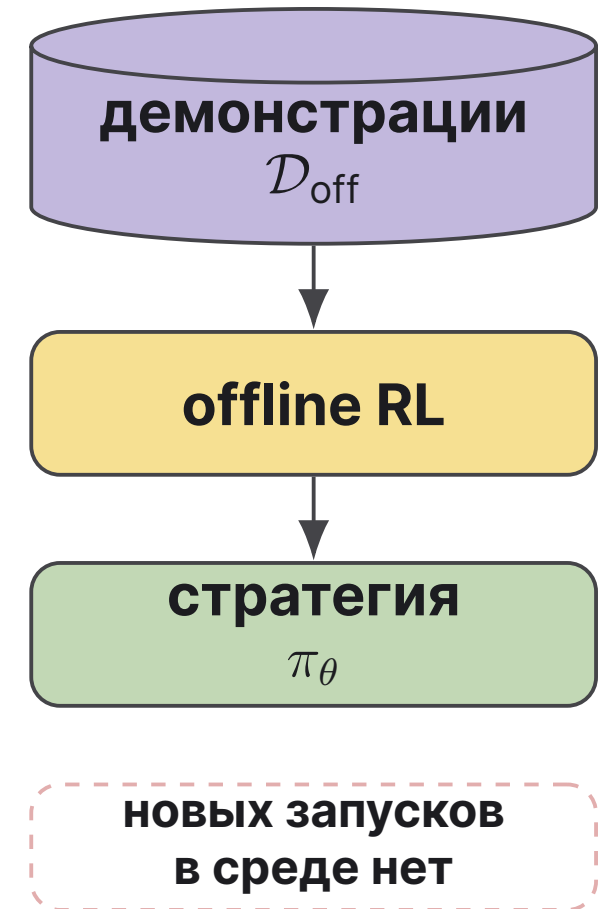


Offline RL

Offline RL: обучение по фиксированному датасету

В offline RL у агента есть только фиксированный датасет:

$$\mathcal{D}_{\text{off}} = \{(o_i, a_i, r_i, o'_i)\}_{i=1}^N.$$

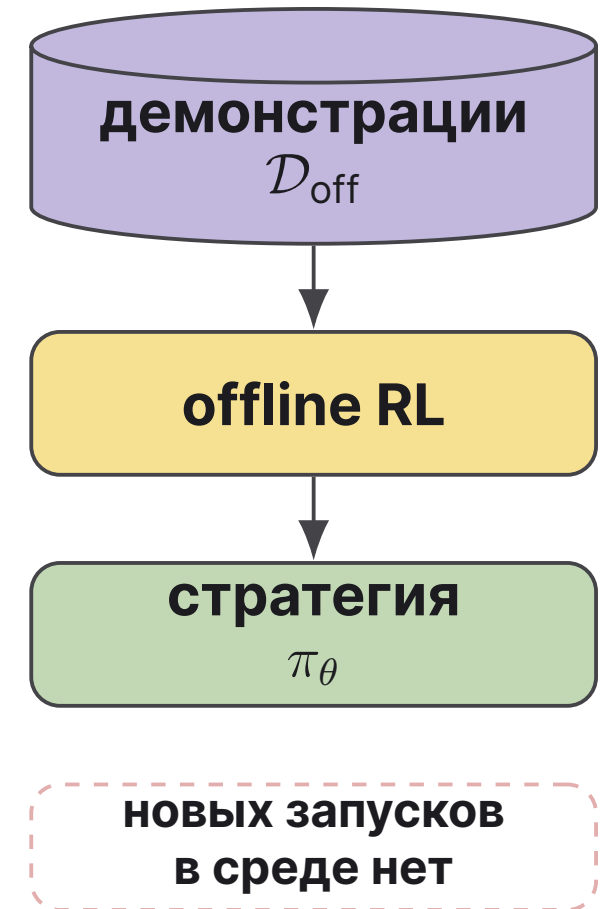


Offline RL: обучение по фиксированному датасету

В offline RL у агента есть только фиксированный датасет:

$$\mathcal{D}_{\text{off}} = \{(o_i, a_i, r_i, o'_i)\}_{i=1}^N.$$

Во время обучения **нет дополнительного взаимодействия со средой.**



Offline RL: обучение по фиксированному датасету

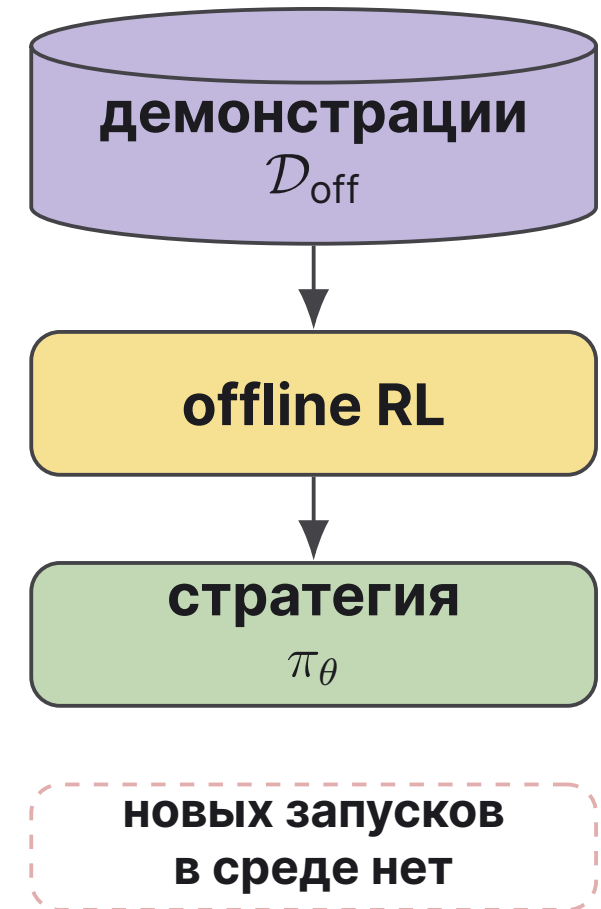
В offline RL у агента есть только фиксированный датасет:

$$\mathcal{D}_{\text{off}} = \{(o_i, a_i, r_i, o'_i)\}_{i=1}^N.$$

Во время обучения **нет дополнительного взаимодействия со средой.**

Задача амбициознее, чем в BC:

обучить стратегию, которая действует лучше
поведения, записанного в датасете.



Offline RL: обучение по фиксированному датасету

В offline RL у агента есть только фиксированный датасет:

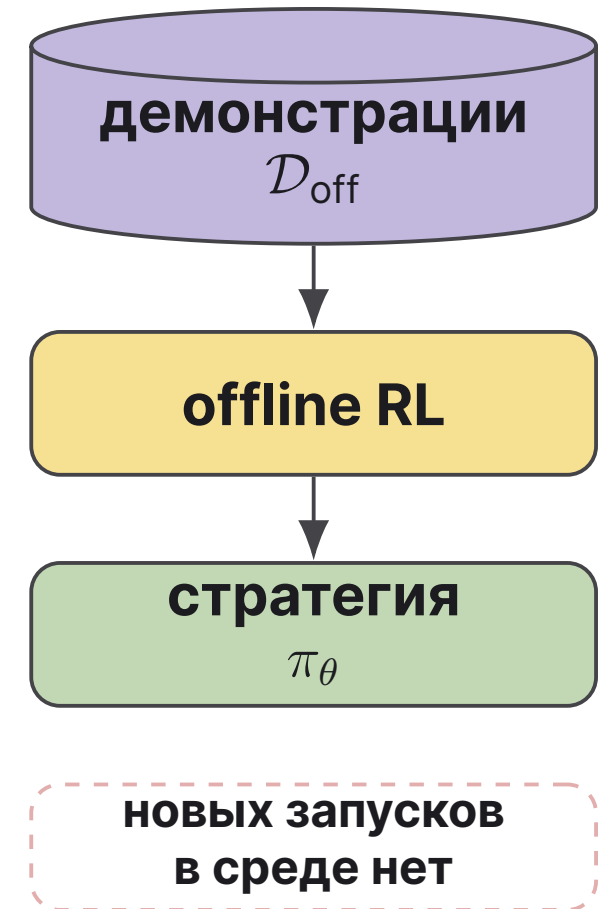
$$\mathcal{D}_{\text{off}} = \{(o_i, a_i, r_i, o'_i)\}_{i=1}^N.$$

Во время обучения **нет дополнительного взаимодействия со средой.**

Задача амбициознее, чем в BC:

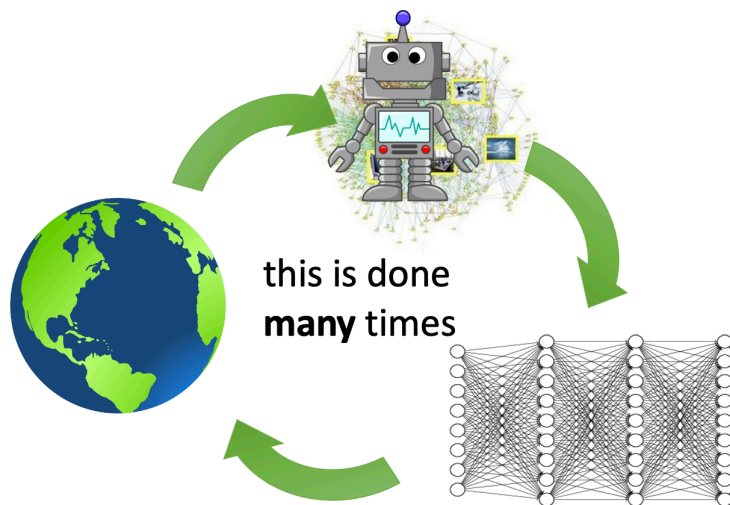
обучить стратегию, которая действует лучше
поведения, записанного в датасете.

Но улучшение опасно: в датасете может быть недостаточное количество переходов для состояния, которые выберет новая стратегия.

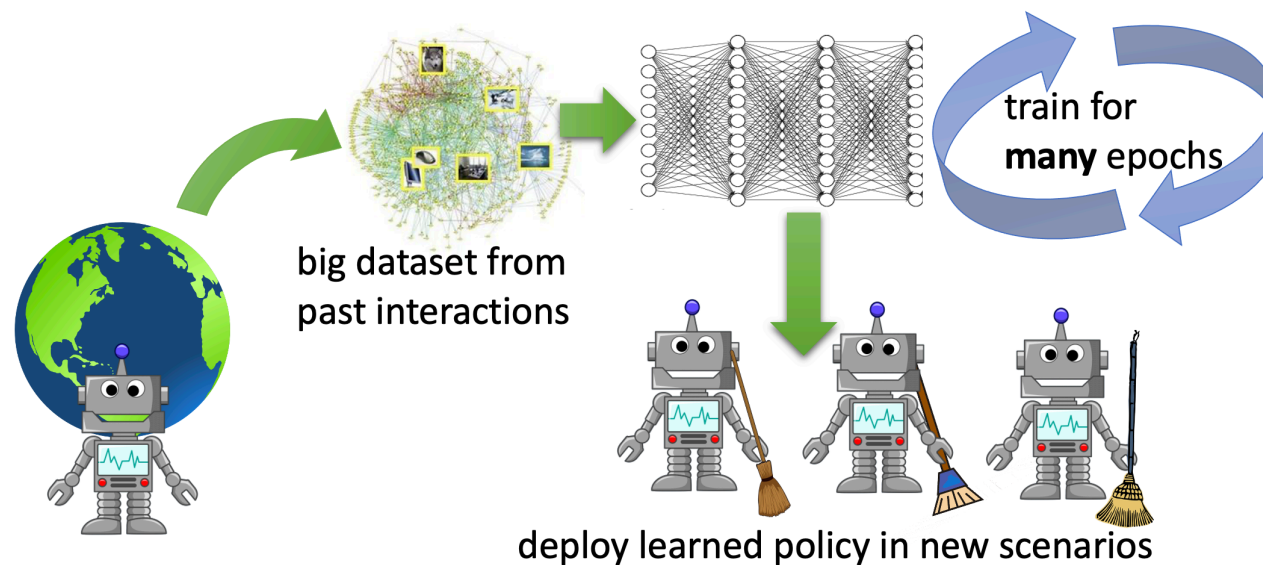


Почему offline RL сложен?

reinforcement learning

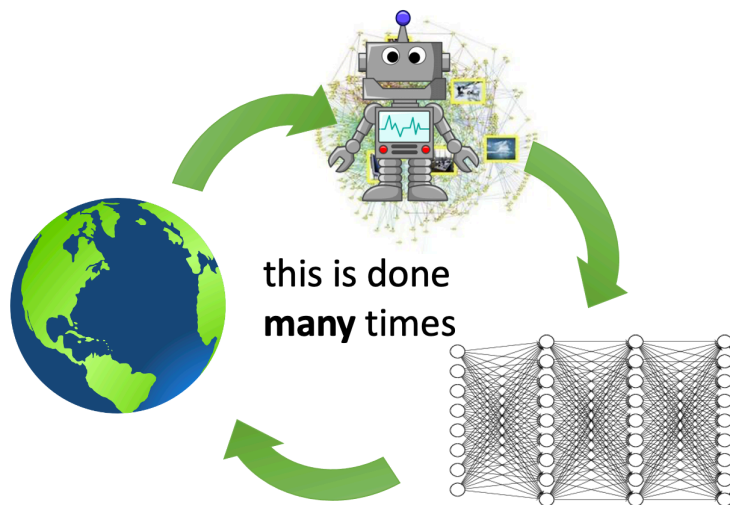


offline reinforcement learning

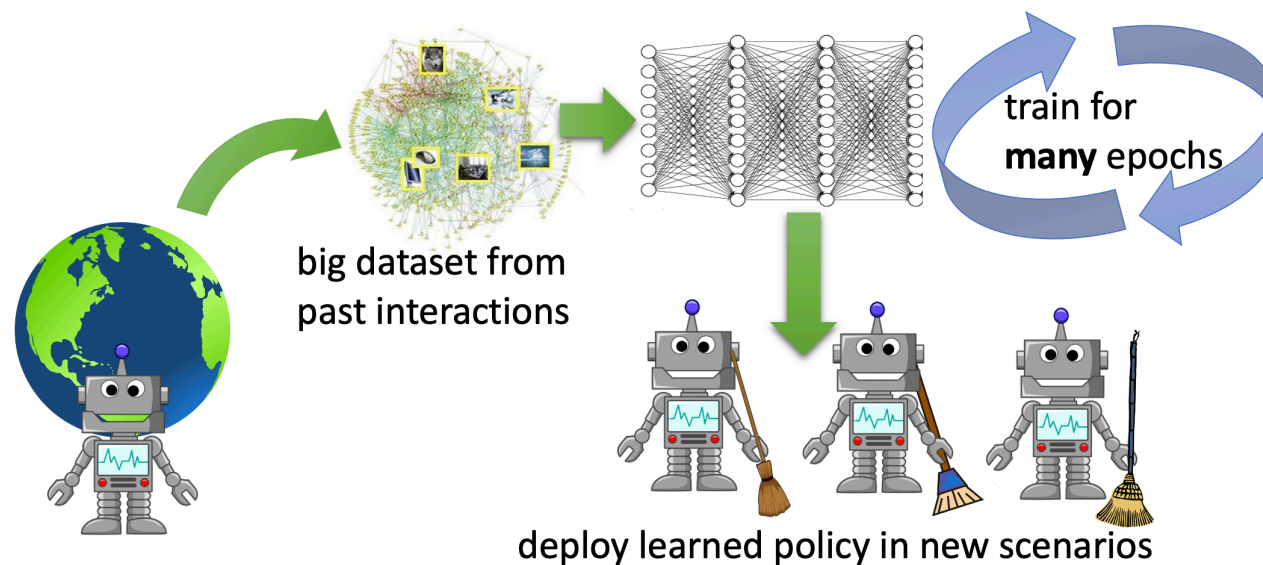


Почему offline RL сложен?

reinforcement learning



offline reinforcement learning



В Online RL постановке — агент может исправлять ошибки, собирая новые переходы.
В Offline RL постановке — не может.

Ключевой режим отказа: ошибка экстраполяции

В стандартном actor-critic улучшение стратегии выглядит примерно так:

$$\pi_{\text{new}}(o) \approx \arg \max_a Q_{\phi}(o, a).$$

Ключевой режим отказа: ошибка экстраполяции

В стандартном actor-critic улучшение стратегии выглядит примерно так:

$$\pi_{\text{new}}(o) \approx \arg \max_a Q_{\phi}(o, a).$$

В offline RL максимизация может выбрать действия, которых нет в датасете:

$$a \not\sim \mathcal{D}_{\text{off}}(a \mid o).$$

Ключевой режим отказа: ошибка экстраполяции

В стандартном actor-critic улучшение стратегии выглядит примерно так:

$$\pi_{\text{new}}(o) \approx \arg \max_a Q_{\phi}(o, a).$$

В offline RL максимизация может выбрать действия, которых нет в датасете:

$$a \not\sim \mathcal{D}_{\text{off}}(a \mid o).$$

Тогда $Q_{\phi}(o, a)$ является экстраполяцией. Если значение завышено, стратегия начинает пользоваться ошибкой оценки, а не реальной полезностью.

Ключевой режим отказа: ошибка экстраполяции

В стандартном actor-critic улучшение стратегии выглядит примерно так:

$$\pi_{\text{new}}(o) \approx \arg \max_a Q_{\phi}(o, a).$$

В offline RL максимизация может выбрать действия, которых нет в датасете:

$$a \not\sim \mathcal{D}_{\text{off}}(a \mid o).$$

Тогда $Q_{\phi}(o, a)$ является экстраполяцией. Если значение завышено, стратегия начинает пользоваться ошибкой оценки, а не реальной полезностью.

Принцип offline RL

Улучшать стратегию только там, где в датасете есть достаточное количество переходов, вне этих данных оценки полезности оставлять консервативными.

Два основных семейства методов

Ограничивать стратегию

- оставаться близко к действиям из датасета
- избегать неподдержанных действий
- примеры: BCQ, BEAR, TD3+BC

Два основных семейства методов

Ограничивать стратегию

- оставаться близко к действиям из датасета
- избегать неподдержанных действий
- примеры: BCQ, BEAR, TD3+BC

Консервативные полезности

- штрафовать высокие значения для OOD-действий
- учить пессимистичные Q-функции
- примеры: CQL, IQL-style избегание невиданных действий

Два основных семейства методов

Ограничивать стратегию

- оставаться близко к действиям из датасета
- избегать неподдержанных действий
- примеры: BCQ, BEAR, TD3+BC

Консервативные полезности

- штрафовать высокие значения для OOD-действий
- учить пессимистичные Q-функции
- примеры: CQL, IQL-style избегание невиданных действий

Оба подхода ограничивают улучшение стратегии: не дают использовать завышенные полезности для пар вне датасета.

CQL: где можно доверять полезностям?

Зафиксируем одно наблюдение o .

Offline-датасет показывает только часть действий:

$$a_{\text{data}} \sim \mathcal{D}_{\text{off}}(\cdot \mid o).$$

CQL: где можно доверять полезностям?

Зафиксируем одно наблюдение o .

Offline-датасет показывает только часть действий:

$$a_{\text{data}} \sim \mathcal{D}_{\text{off}}(\cdot \mid o).$$

- там, где датасет покрывает действия, Q_ϕ можно учить по переходам
- там, где покрытия нет, Q_ϕ легко становится чрезмерно оптимистичной
- CQL делает такие оценки консервативными

CQL: где можно доверять полезностям?

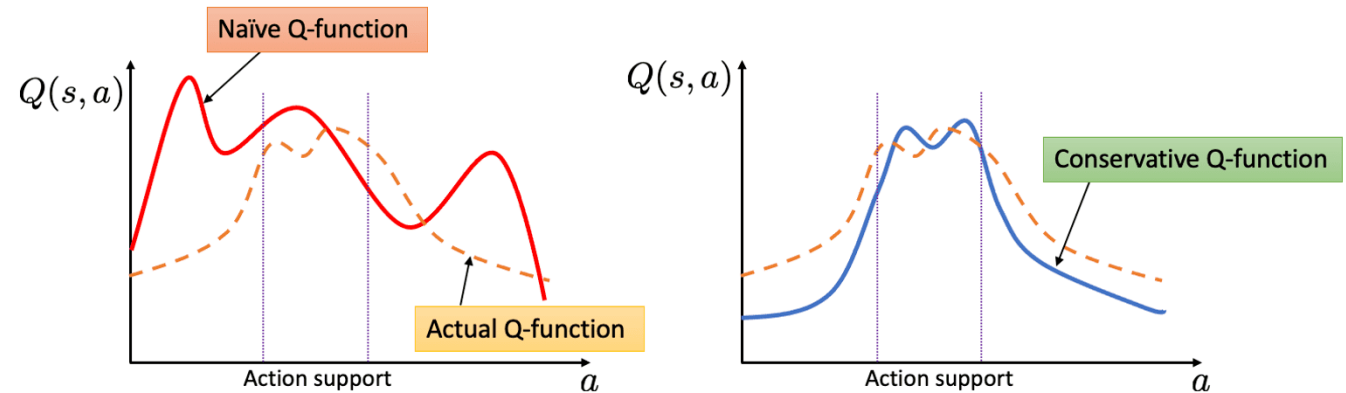
Зафиксируем одно наблюдение o .

Offline-датасет показывает только часть действий:

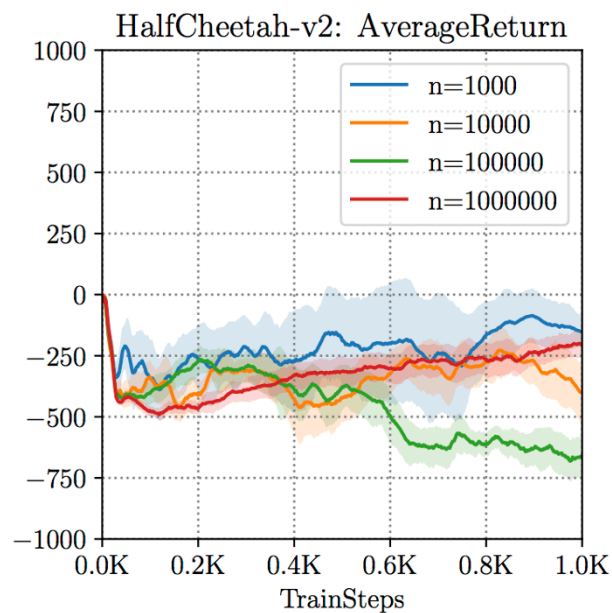
$$a_{\text{data}} \sim \mathcal{D}_{\text{off}}(\cdot \mid o).$$

- там, где датасет покрывает действия, Q_ϕ можно учить по переходам
- там, где покрытия нет, Q_ϕ легко становится чрезмерно оптимистичной
- CQL делает такие оценки консервативными

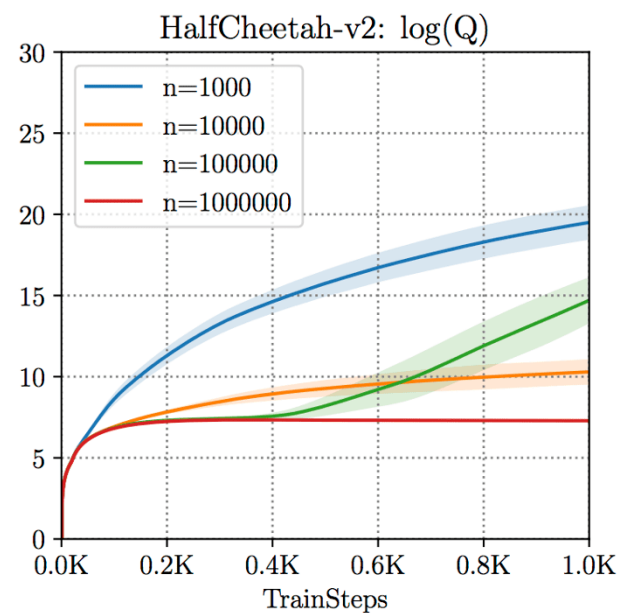
Идея CQL: штрафовать необоснованно высокий $Q_\phi(o, a)$ вне поддержки датасета.



CQL: когда оценка растёт быстрее качества



How well it does



How well it thinks it does

Цель CQL: кандидаты против данных

Упрощенно: обычный Bellman loss + консервативный штраф для каждого o из датасета.

$$\mathcal{L}_{\text{CQL}}(\phi) = \mathcal{L}_{\text{Bellman}}(\phi) + \alpha \mathbb{E}_{o \sim \mathcal{D}_{\text{off}}} [\text{penalty}(o)]$$

Цель CQL: кандидаты против данных

Упрощенно: обычный Bellman loss + консервативный штраф для каждого o из датасета.

$$\mathcal{L}_{\text{CQL}}(\phi) = \mathcal{L}_{\text{Bellman}}(\phi) + \alpha \mathbb{E}_{o \sim \mathcal{D}_{\text{off}}} [\text{penalty}(o)]$$

$$\text{penalty}(o) = \underbrace{\log \sum_a \exp Q_\phi(o, a)}_{\text{кандидаты: штраф за высокие } Q} - \underbrace{\mathbb{E}_{a \sim \mathcal{D}_{\text{off}}(\cdot|o)} Q_\phi(o, a)}_{\text{данные: сохранить высокие } Q}.$$

Цель CQL: кандидаты против данных

Упрощенно: обычный Bellman loss + консервативный штраф для каждого o из датасета.

$$\mathcal{L}_{\text{CQL}}(\phi) = \mathcal{L}_{\text{Bellman}}(\phi) + \alpha \mathbb{E}_{o \sim \mathcal{D}_{\text{off}}} [\text{penalty}(o)]$$

$$\text{penalty}(o) = \underbrace{\log \sum_a \exp Q_\phi(o, a)}_{\text{кандидаты: штраф за высокие } Q} - \underbrace{\mathbb{E}_{a \sim \mathcal{D}_{\text{off}}(\cdot|o)} Q_\phi(o, a)}_{\text{данные: сохранить высокие } Q}.$$

- $\log \sum \exp Q$ — мягкий максимум: один слишком высокий кандидат уже увеличивает loss.
- Вычитание Q для действий из данных не позволяет просто занизить все Q сразу.
- α управляет консервативностью: больше α — меньше доверия к действиям вне данных.

CQL: зачем сравнивать действия-кандидаты с действиями из датасета?

Так как мы **минимизируем** функцию потерь, штраф CQL удобно читать так:

$$\min_Q \left(Q(\text{кандидаты}) - Q(\text{действия из датасета}) \right).$$

CQL: зачем сравнивать действия-кандидаты с действиями из датасета?

Так как мы **минимизируем** функцию потерь, штраф CQL удобно читать так:

$$\min_Q \left(Q(\text{кандидаты}) - Q(\text{действия из датасета}) \right).$$

Оптимизатор уменьшает этот штраф двумя способами:

- снизить Q для кандидатов, особенно для действий вне датасета;
- не занижать Q для действий из датасета: они входят в штраф со знаком минус.

CQL: зачем сравнивать действия-кандидаты с действиями из датасета?

Так как мы **минимизируем** функцию потерь, штраф CQL удобно читать так:

$$\min_Q \left(Q(\text{кандидаты}) - Q(\text{действия из датасета}) \right).$$

Оптимизатор уменьшает этот штраф двумя способами:

- снизить Q для кандидатов, особенно для действий вне датасета;
- не занижать Q для действий из датасета: они входят в штраф со знаком минус.

Интуиция минуса

Не нужно делать все Q маленькими. Нужно, чтобы неподдержанные кандидаты получали меньшие значения, чем действия, реально наблюдавшиеся в датасете.

Практические обновления актора

IQL не оценивает действия стратегии, которых нет в данных. Он извлекает стратегию через взвешенное поведенческое клонирование:

$$\max_{\theta} \mathbb{E}_{(o,a) \sim \mathcal{D}_{\text{off}}} \left[\exp\left(\frac{A(o,a)}{\beta}\right) \log \pi_{\theta}(a \mid o) \right].$$

Практические обновления актора

IQL не оценивает действия стратегии, которых нет в данных. Он извлекает стратегию через взвешенное поведенческое клонирование:

$$\max_{\theta} \mathbb{E}_{(o,a) \sim \mathcal{D}_{\text{off}}} \left[\exp\left(\frac{A(o,a)}{\beta}\right) \log \pi_{\theta}(a \mid o) \right].$$

AWAC: похожая идея: сильнее имитировать действия из датасета, при высоком преимуществе.

Практические обновления актора

IQL не оценивает действия стратегии, которых нет в данных. Он извлекает стратегию через взвешенное поведенческое клонирование:

$$\max_{\theta} \mathbb{E}_{(o,a) \sim \mathcal{D}_{\text{off}}} \left[\exp\left(\frac{A(o,a)}{\beta}\right) \log \pi_{\theta}(a \mid o) \right].$$

AWAC: похожая идея: сильнее имитировать действия из датасета, при высоком преимуществе.

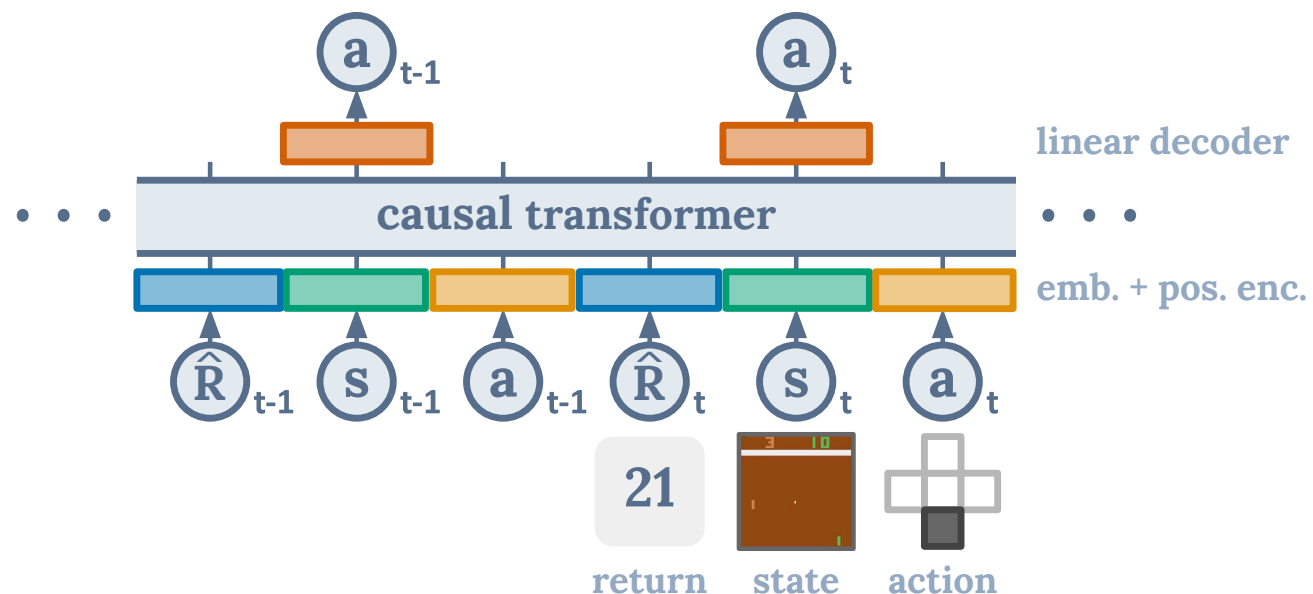
TD3+BC: сделать обновление актора простым:

$$\max_{\theta} \mathbb{E}_{o \sim \mathcal{D}_{\text{off}}} \left[Q_{\phi}(o, \pi_{\theta}(o)) - \lambda \|\pi_{\theta}(o) - a_{\mathcal{D}_{\text{off}}}\|_2^2 \right].$$

Decision Transformer: RL как последовательность

- ▶ Траектория записывается как последовательность токенов:

$$(\hat{R}_1, o_1, a_1, \hat{R}_2, o_2, a_2, \dots).$$

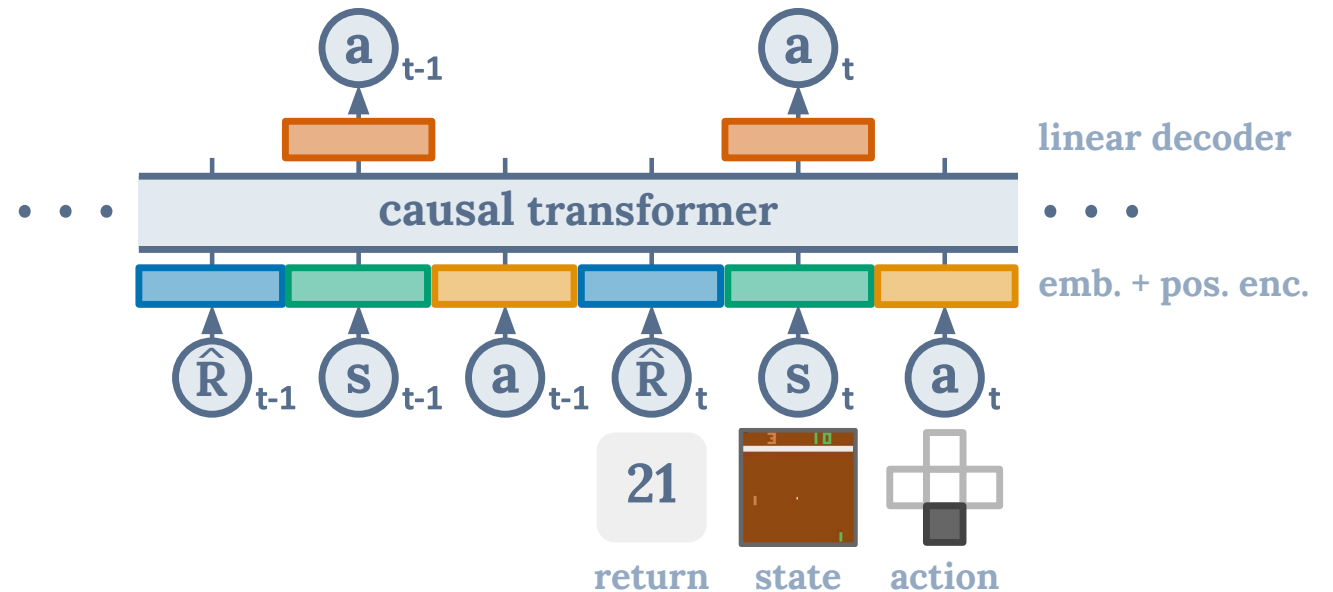


Decision Transformer: RL как последовательность

- Траектория записывается как последовательность токенов:

$$(\hat{R}_1, o_1, a_1, \hat{R}_2, o_2, a_2, \dots).$$

- Causal Transformer учится предсказывать следующее действие.

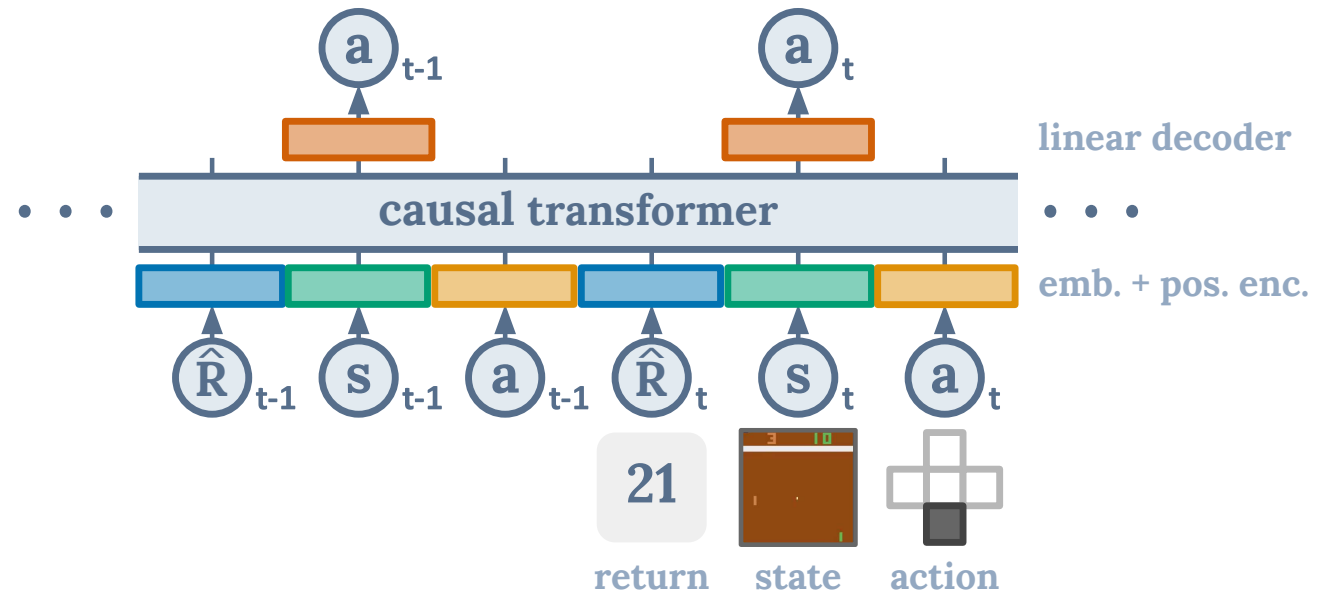


Decision Transformer: RL как последовательность

- Траектория записывается как последовательность токенов:

$$(\hat{R}_1, o_1, a_1, \hat{R}_2, o_2, a_2, \dots).$$

- Causal Transformer учится предсказывать следующее действие.
- Желаемая отдача становится условием для генерации поведения.



Decision Transformer: обучение и применение

$$\log \pi_{\theta}(a_t \mid \hat{R}_{\leq t}, o_{\leq t}, a_{< t})$$

Обучение

- teacher forcing на offline-траекториях
- по прошлому контексту предсказываем a_t

Decision Transformer: обучение и применение

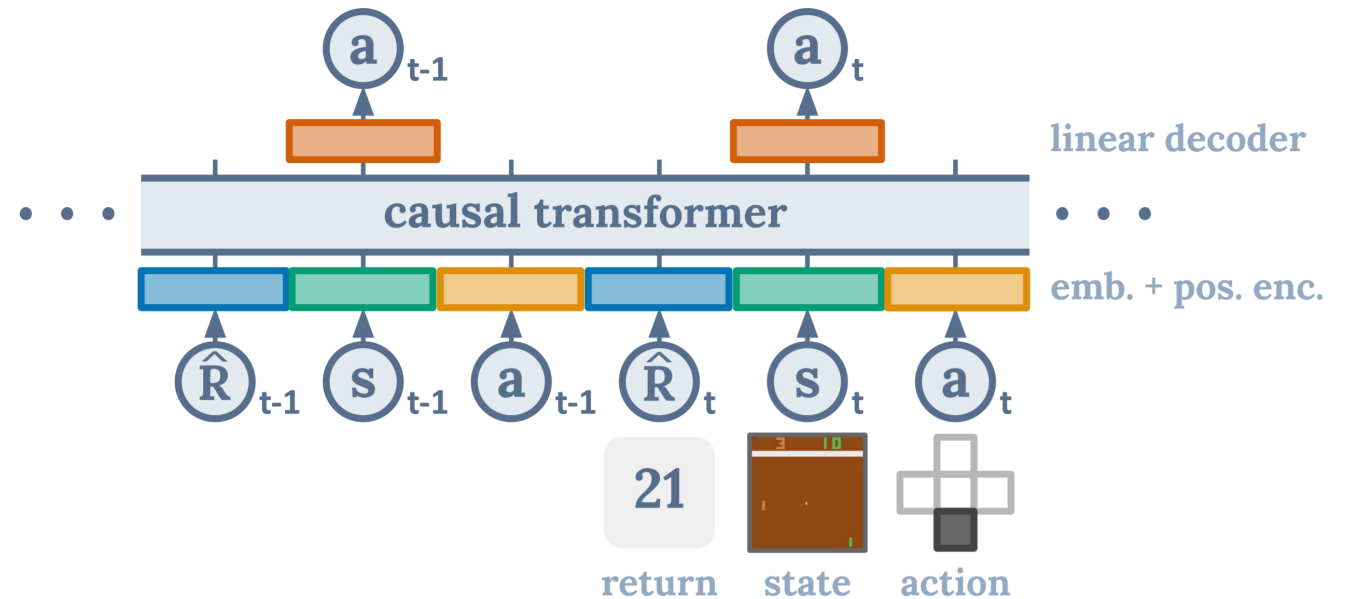
$$\log \pi_{\theta}(a_t \mid \hat{R}_{\leq t}, o_{\leq t}, a_{< t})$$

Обучение

- teacher forcing на offline-траекториях
- по прошлому контексту предсказываем a_t

Применение

- задаем target return $\hat{R}_1$
- наблюдаем o_t и генерируем a_t



Offline RL: что запомнить

- Offline RL учится по фиксированному датасету без взаимодействия во время обучения.

Offline RL: что запомнить

- Offline RL учится по фиксированному датасету без взаимодействия во время обучения.
- Центральный риск — выбор действий, чьи значения не поддержаны данными.

Offline RL: что запомнить

- Offline RL учится по фиксированному датасету без взаимодействия во время обучения.
- Центральный риск — выбор действий, чьи значения не поддержаны данными.
- Классический offline RL контролирует это ограничениями стратегии или консервативными Q-функциями.

Offline RL: что запомнить

- Offline RL учится по фиксированному датасету без взаимодействия во время обучения.
- Центральный риск — выбор действий, чьи значения не поддержаны данными.
- Классический offline RL контролирует это ограничениями стратегии или консервативными Q-функциями.

Offline RL: что запомнить

- Offline RL учится по фиксированному датасету без взаимодействия во время обучения.
- Центральный риск — выбор действий, чьи значения не поддержаны данными.
- Классический offline RL контролирует это ограничениями стратегии или консервативными Q-функциями.

Связь с демонстрациями

Если датасет в основном содержит экспертное поведение, offline RL становится способом улучшить стратегию относительно простой имитации, оставаясь в области, покрытой демонстрациями.

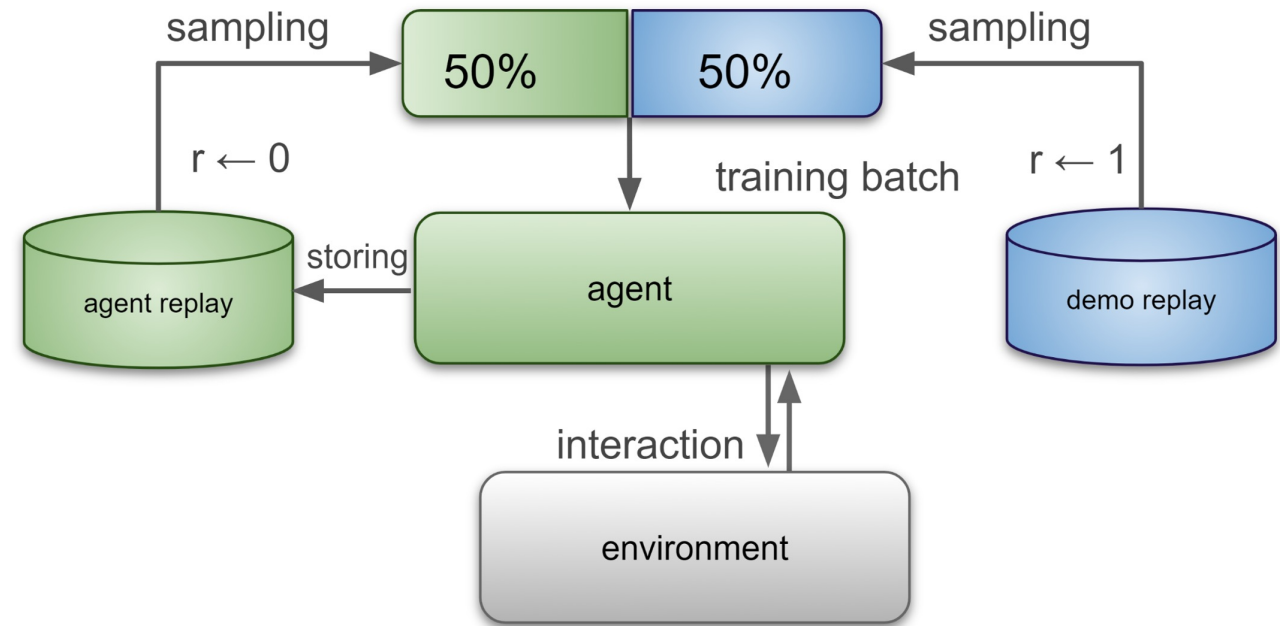
04

Обучение по экспертному поведению в симуляторе

Soft-Q Imitation Learning (SQIL)

Идея:

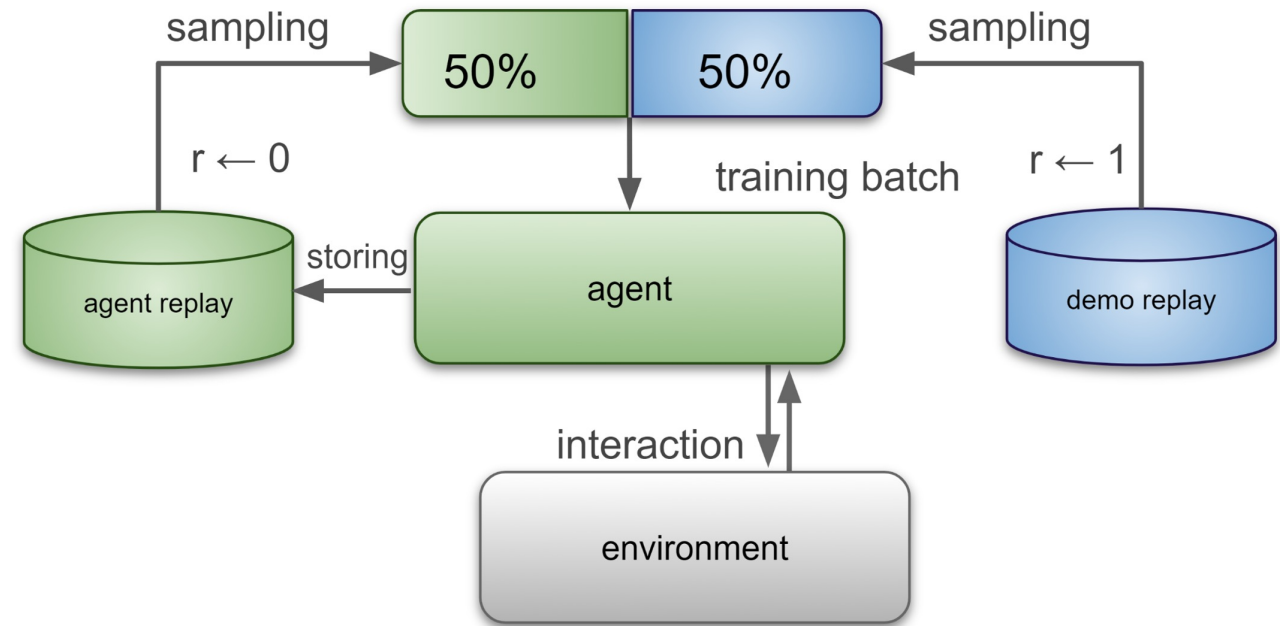
- ▶ первоначально заполняем память прецедентов демонстрациями с вознаграждением $r = +1$;



Soft-Q Imitation Learning (SQIL)

Идея:

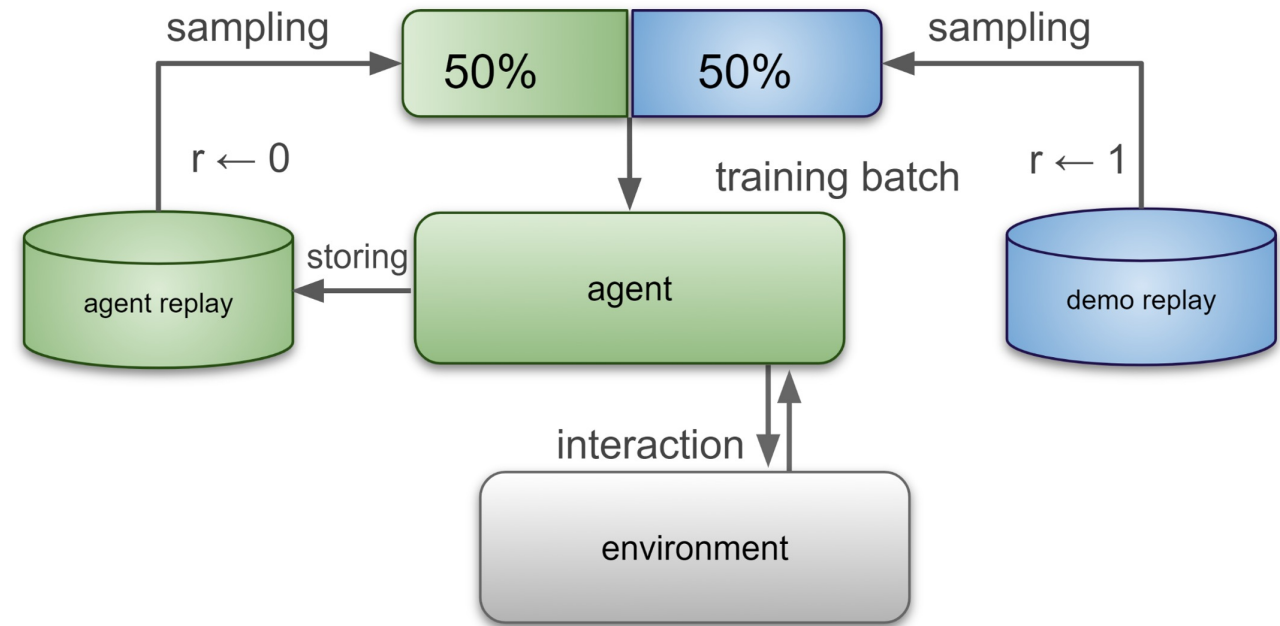
- первоначально заполняем память прецедентов демонстрациями с вознаграждением $r = +1$;
- при взаимодействии агента со средой новые данные добавляются в память прецедентов с наградой $r = 0$;



Soft-Q Imitation Learning (SQL)

Идея:

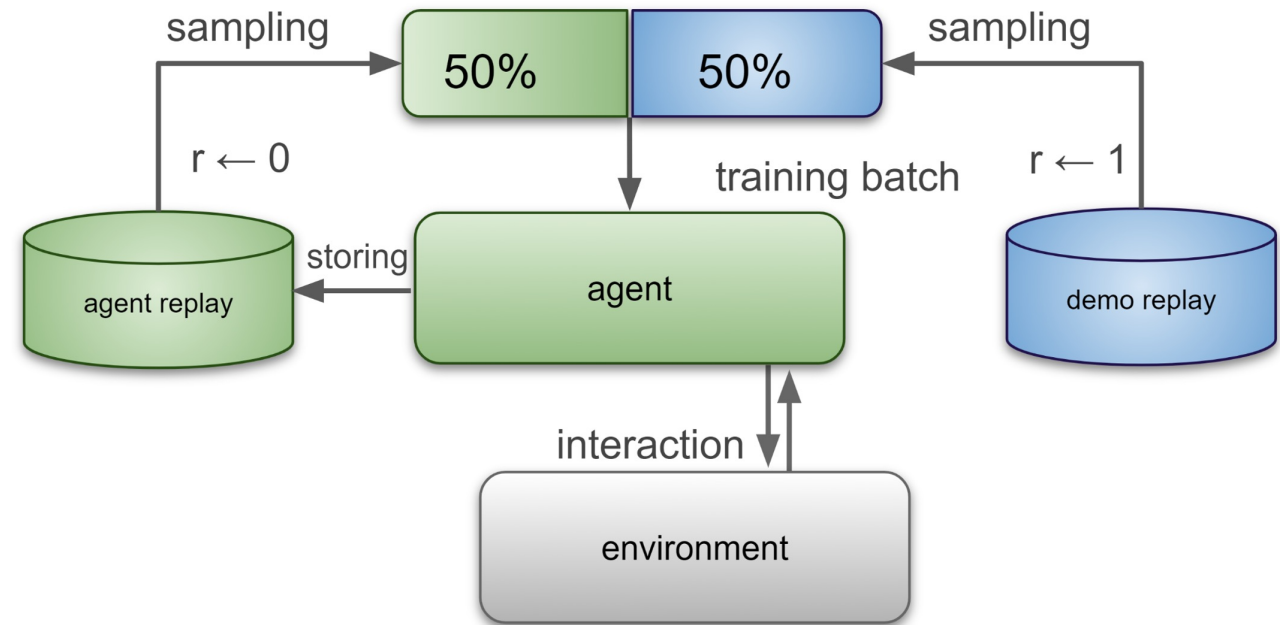
- первоначально заполняем память прецедентов демонстрациями с вознаграждением $r = +1$;
- при взаимодействии агента со средой новые данные добавляются в память прецедентов с наградой $r = 0$;
- в батче поддерживается баланс между демонстрационными и новыми данными: 50% каждого типа.



Soft-Q Imitation Learning (SQL)

Идея:

- первоначально заполняем память прецедентов демонстрациями с вознаграждением $r = +1$;
- при взаимодействии агента со средой новые данные добавляются в память прецедентов с наградой $r = 0$;
- в батче поддерживается баланс между демонстрационными и новыми данными: 50% каждого типа.



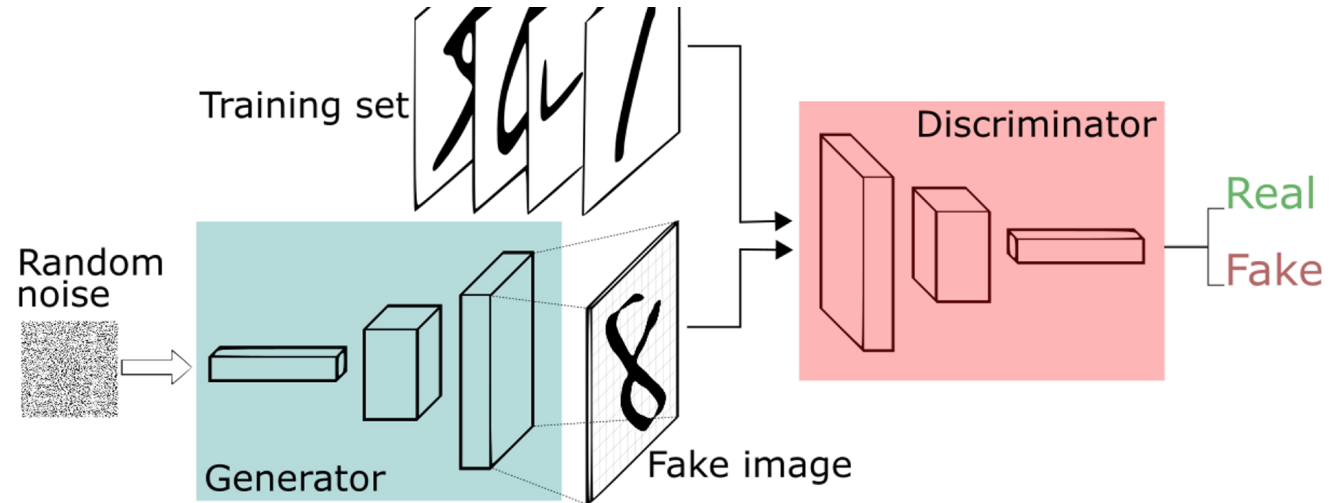
Интерпретация

SQL превращает демонстрации в простую задачу с разреженной наградой: экспертные переходы выглядят полезными, а собственные переходы агента получают нулевую награду.

Recap: Generative Adversarial Imitation Learning

Напоминание про GAN:

- генератор создает синтетические примеры;
- дискриминатор отличает настоящие данные от сгенерированных;
- генератор улучшается, когда дискриминатору становится сложнее отличать сгенерированные примеры от настоящих.



Generative Adversarial Imitation Learning (GAIL)

В GAIL роль генератора играет стратегия агента:

$$\tau_i \sim \pi_\theta.$$

Generative Adversarial Imitation Learning (GAIL)

В GAIL роль генератора играет стратегия агента:

$$\tau_i \sim \pi_\theta.$$

Роль настоящих данных играют экспертные траектории:

$$\tau_E \sim \pi_E.$$

Generative Adversarial Imitation Learning (GAIL)

В GAIL роль генератора играет стратегия агента:

$$\tau_i \sim \pi_\theta.$$

Роль настоящих данных играют экспертные траектории:

$$\tau_E \sim \pi_E.$$

Дискриминатор $D_\phi(s_t, a_t)$ оценивает, что пара состояние-действие получена от агента, а не из демонстраций эксперта.

Generative Adversarial Imitation Learning (GAIL)

В GAIL роль генератора играет стратегия агента:

$$\tau_i \sim \pi_\theta.$$

Роль настоящих данных играют экспертные траектории:

$$\tau_E \sim \pi_E.$$

Дискриминатор $D_\phi(s_t, a_t)$ оценивает, что пара состояние-действие получена от агента, а не из демонстраций эксперта.

Состязательная игра

- D_ϕ учится отличать траектории агента от экспертных;
- π_θ учится делать свои траектории неотличимыми от экспертных;
- награда получается из дискриминатора, а не задается вручную.

Алгоритм GAIL

Вход: экспертные траектории $\mathcal{D}_E$, начальная стратегия π_θ , дискриминатор D_ϕ

1: **for** $i = 0, 1, 2, \dots$ **do**

8: **end for**

Алгоритм GAIL

Вход: экспертные траектории $\mathcal{D}_E$, начальная стратегия π_θ , дискриминатор D_ϕ

1: **for** $i = 0, 1, 2, \dots$ **do**

2: Запустить π_θ в симуляторе и собрать траектории $\tau_i \sim \pi_\theta$

8: **end for**

Алгоритм GAIL

Вход: экспертные траектории $\mathcal{D}_E$, начальная стратегия π_θ , дискриминатор D_ϕ

- 1: **for** $i = 0, 1, 2, \dots$ **do**
- 2: Запустить π_θ в симуляторе и собрать траектории $\tau_i \sim \pi_\theta$
- 3: Составить батчи $B_\pi = \{(s_t, a_t) \sim \tau_i\}$ и $B_E = \{(s_t, a_t) \sim \mathcal{D}_E\}$
- 8: **end for**

Алгоритм GAIL

Вход: экспертные траектории $\mathcal{D}_E$, начальная стратегия π_θ , дискриминатор D_ϕ

- 1: **for** $i = 0, 1, 2, \dots$ **do**
- 2: Запустить π_θ в симуляторе и собрать траектории $\tau_i \sim \pi_\theta$
- 3: Составить батчи $B_\pi = \{(s_t, a_t) \sim \tau_i\}$ и $B_E = \{(s_t, a_t) \sim \mathcal{D}_E\}$
- 4: $\mathcal{L}_D(\phi) \leftarrow -\frac{1}{|B_\pi|} \sum_{B_\pi} \log D_\phi(s_t, a_t) - \frac{1}{|B_E|} \sum_{B_E} \log(1 - D_\phi(s_t, a_t))$
- 8: **end for**

Алгоритм GAIL

Вход: экспертные траектории $\mathcal{D}_E$, начальная стратегия π_θ , дискриминатор D_ϕ

- 1: **for** $i = 0, 1, 2, \dots$ **do**
- 2: Запустить π_θ в симуляторе и собрать траектории $\tau_i \sim \pi_\theta$
- 3: Составить батчи $B_\pi = \{(s_t, a_t) \sim \tau_i\}$ и $B_E = \{(s_t, a_t) \sim \mathcal{D}_E\}$
- 4: $\mathcal{L}_D(\phi) \leftarrow -\frac{1}{|B_\pi|} \sum_{B_\pi} \log D_\phi(s_t, a_t) - \frac{1}{|B_E|} \sum_{B_E} \log(1 - D_\phi(s_t, a_t))$
- 5: $\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}_D(\phi)$ ▷ дискриминатор
- 8: **end for**

Алгоритм GAIL

Вход: экспертные траектории $\mathcal{D}_E$, начальная стратегия π_θ , дискриминатор D_ϕ

- 1: **for** $i = 0, 1, 2, \dots$ **do**
- 2: Запустить π_θ в симуляторе и собрать траектории $\tau_i \sim \pi_\theta$
- 3: Составить батчи $B_\pi = \{(s_t, a_t) \sim \tau_i\}$ и $B_E = \{(s_t, a_t) \sim \mathcal{D}_E\}$
- 4: $\mathcal{L}_D(\phi) \leftarrow -\frac{1}{|B_\pi|} \sum_{B_\pi} \log D_\phi(s_t, a_t) - \frac{1}{|B_E|} \sum_{B_E} \log(1 - D_\phi(s_t, a_t))$
- 5: $\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}_D(\phi)$
- 6: $\hat{c}_\phi(s_t, a_t) \leftarrow \log D_\phi(s_t, a_t)$
- 8: **end for**

▷ дискриминатор
▷ СТОИМОСТЬ

Алгоритм GAIL

Вход: экспертные траектории $\mathcal{D}_E$, начальная стратегия π_θ , дискриминатор D_ϕ

- 1: **for** $i = 0, 1, 2, \dots$ **do**
- 2: Запустить π_θ в симуляторе и собрать траектории $\tau_i \sim \pi_\theta$
- 3: Составить батчи $B_\pi = \{(s_t, a_t) \sim \tau_i\}$ и $B_E = \{(s_t, a_t) \sim \mathcal{D}_E\}$
- 4: $\mathcal{L}_D(\phi) \leftarrow -\frac{1}{|B_\pi|} \sum_{B_\pi} \log D_\phi(s_t, a_t) - \frac{1}{|B_E|} \sum_{B_E} \log(1 - D_\phi(s_t, a_t))$
- 5: $\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}_D(\phi)$
- 6: $\hat{c}_\phi(s_t, a_t) \leftarrow \log D_\phi(s_t, a_t)$
- 7: Обновить θ шагом градиента стратегии или TRPO
- 8: **end for**

▷ дискриминатор
▷ стоимость
▷ стратегия

GAIL: обновление дискриминатора

Дискриминатор обучается отличать траектории агента от данных эксперта:

$$\max_{\phi} \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta}} [\log D_{\phi}(s_t, a_t)] + \mathbb{E}_{(s_t, a_t) \sim \pi_E} [\log(1 - D_{\phi}(s_t, a_t))].$$

GAIL: обновление дискриминатора

Дискриминатор обучается отличать траектории агента от данных эксперта:

$$\max_{\phi} \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta}} [\log D_{\phi}(s_t, a_t)] + \mathbb{E}_{(s_t, a_t) \sim \pi_E} [\log(1 - D_{\phi}(s_t, a_t))].$$

- первая часть использует состояния и действия, полученные текущим агентом;
- вторая часть использует экспертные демонстрации;
- оптимизация обновляет параметры ϕ дискриминатора.

GAIL: обновление дискриминатора

Дискриминатор обучается отличать траектории агента от данных эксперта:

$$\max_{\phi} \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta}} [\log D_{\phi}(s_t, a_t)] + \mathbb{E}_{(s_t, a_t) \sim \pi_E} [\log(1 - D_{\phi}(s_t, a_t))].$$

- первая часть использует состояния и действия, полученные текущим агентом;
- вторая часть использует экспертные демонстрации;
- оптимизация обновляет параметры ϕ дискриминатора.

Если $D_{\phi}(s_t, a_t)$ велик, дискриминатор считает переход похожим на данные агента. Если мал, переход выглядит экспертным.

GAIL: обновление стратегии

После обновления дискриминатора стратегия делает шаг градиента стратегии или TRPO-шаг. Стоимость перехода берется из дискриминатора:

$$\hat{c}_\phi(s_t, a_t) = \log D_\phi(s_t, a_t).$$

GAIL: обновление стратегии

После обновления дискриминатора стратегия делает шаг градиента стратегии или TRPO-шаг. Стоимость перехода берется из дискриминатора:

$$\hat{c}_\phi(s_t, a_t) = \log D_\phi(s_t, a_t).$$

Обновление стратегии можно читать как обычный градиент стратегии с выученной стоимостью:

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{(s_t, a_t) \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \hat{Q}_t \right] - \lambda \nabla_\theta \mathcal{H}[\pi_\theta].$$

GAIL: обновление стратегии

После обновления дискриминатора стратегия делает шаг градиента стратегии или TRPO-шаг. Стоимость перехода берется из дискриминатора:

$$\hat{c}_\phi(s_t, a_t) = \log D_\phi(s_t, a_t).$$

Обновление стратегии можно читать как обычный градиент стратегии с выученной стоимостью:

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{(s_t, a_t) \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \hat{Q}_t \right] - \lambda \nabla_\theta \mathcal{H}[\pi_\theta].$$

Стратегия обновляется так, чтобы ее траектории становились менее отличимыми от экспертных.

SQIL и GAIL: обучение через взаимодействие

SQIL

- демонстрации получают фиксированную награду $+1$
- новые переходы агента получают 0
- дальше используется алгоритм обучения с подкреплением вне текущей стратегии

SQL и GAIL: обучение через взаимодействие

SQL

- демонстрации получают фиксированную награду $+1$
- новые переходы агента получают 0
- дальше используется алгоритм обучения с подкреплением вне текущей стратегии

GAIL

- награда не задается вручную
- дискриминатор учит различать агента и эксперта
- стратегия обучается делать траектории неотличимыми от экспертных

SQL и GAIL: обучение через взаимодействие

SQL

- демонстрации получают фиксированную награду $+1$
- новые переходы агента получают 0
- дальше используется алгоритм обучения с подкреплением вне текущей стратегии

GAIL

- награда не задается вручную
- дискриминатор учит различать агента и эксперта
- стратегия обучается делать траектории неотличимыми от экспертных

Общее свойство

В обоих случаях нужны демонстрации и запуск агента в среде или симуляторе: агент собирает собственные переходы, а обучение связывает их с экспертным поведением.

05

Обучение по демонстрациям
в среде

DQfD:

$$r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \max_a \gamma^n Q(s_{t+n}, a)$$

N-Step Return

$$J(Q) = J_{DQ}(Q) + \lambda_1 J_n(Q) + \lambda_2 J_E(Q) + \lambda_3 J_{L2}(Q)$$

margin

L2 regularization

$$J_E(Q) = \max_{a \in A} [Q(s, a) + l(a_E, a)] - Q(s, a_E)$$

Expert margin $l(a_E, a)$ when $a \neq a_E = 0.8$

Algorithm 1 Deep Q-learning from Demonstrations.

- 1: Inputs: $\mathcal{D}^{replay}$: initialized with demonstration data set, θ : weights for initial behavior network (random), θ' : weights for target network (random), τ : frequency at which to update target net, k : number of pre-training gradient updates
 - 2: **for** steps $t \in \{1, 2, \dots, k\}$ **do**
 - 3: Sample a mini-batch of n transitions from $\mathcal{D}^{replay}$ with prioritization
 - 4: Calculate loss $J(Q)$ using target network
 - 5: Perform a gradient descent step to update θ
 - 6: **if** $t \bmod \tau = 0$ **then** $\theta' \leftarrow \theta$ **end if**
 - 7: **end for**
 - 8: **for** steps $t \in \{1, 2, \dots\}$ **do**
 - 9: Sample action from behavior policy $a \sim \pi^{\epsilon_{Q_\theta}}$
 - 10: Play action a and observe (s', r) .
 - 11: Store (s, a, r, s') into $\mathcal{D}^{replay}$, overwriting oldest self-generated transition if over capacity
 - 12: Sample a mini-batch of n transitions from $\mathcal{D}^{replay}$ with prioritization
 - 13: Calculate loss $J(Q)$ using target network
 - 14: Perform a gradient descent step to update θ
 - 15: **if** $t \bmod \tau = 0$ **then** $\theta' \leftarrow \theta$ **end if**
 - 16: $s \leftarrow s'$
 - 17: **end for**
-

R2D3:

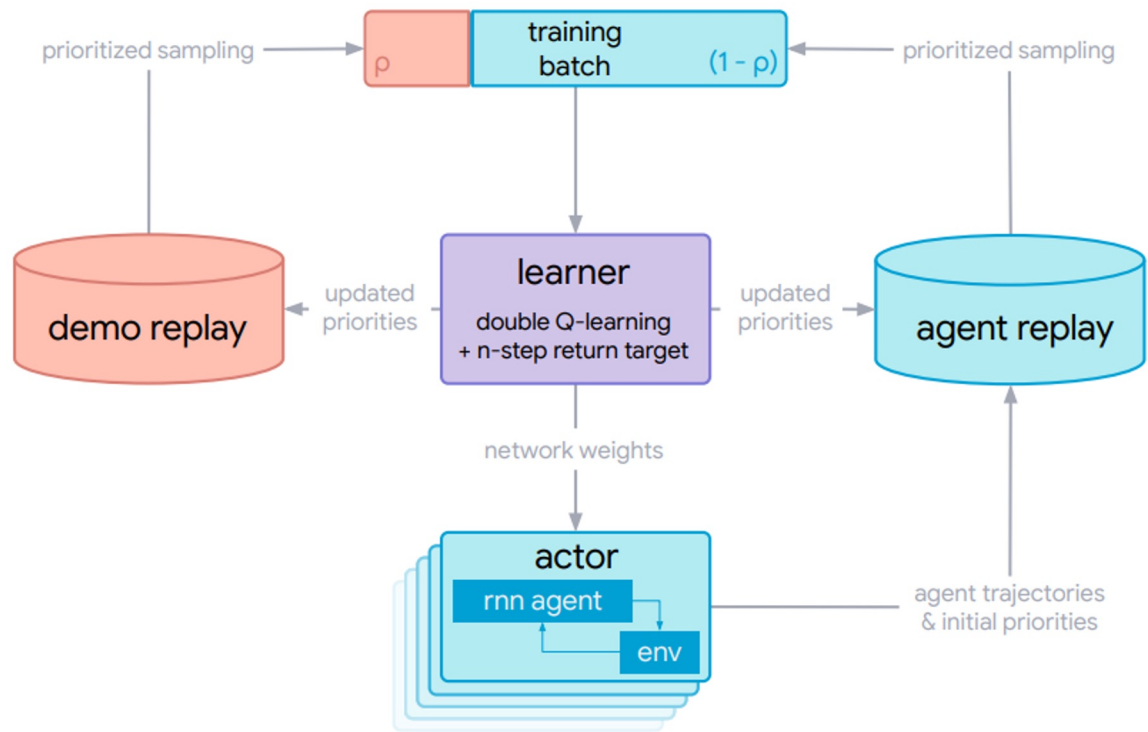


Figure 1 | The R2D3 distributed system diagram. The learner samples batches that are a mixture of demonstrations and the experiences the agent generates by interacting with the environment over the course of training. The ratio between demos and agent experiences is a key hyper-parameter which must be carefully tuned to achieve good performance.

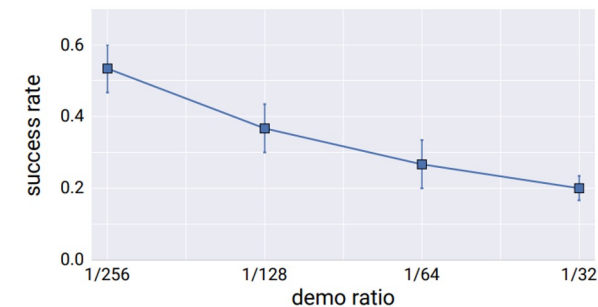
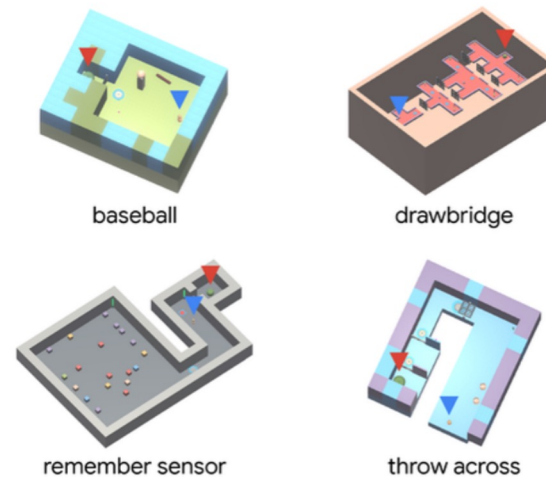
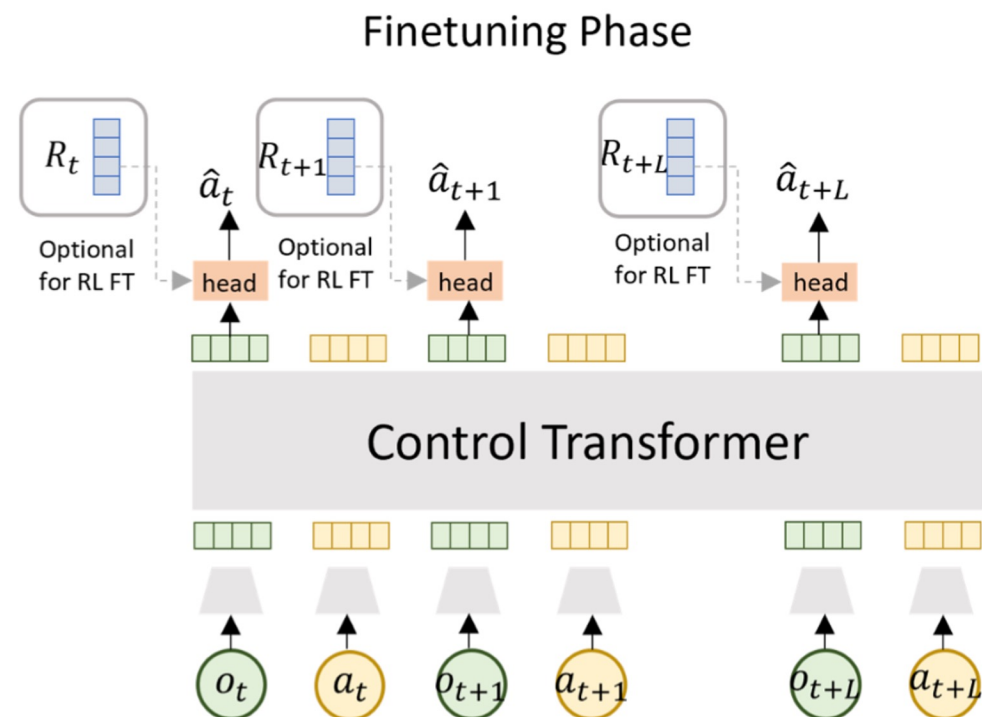
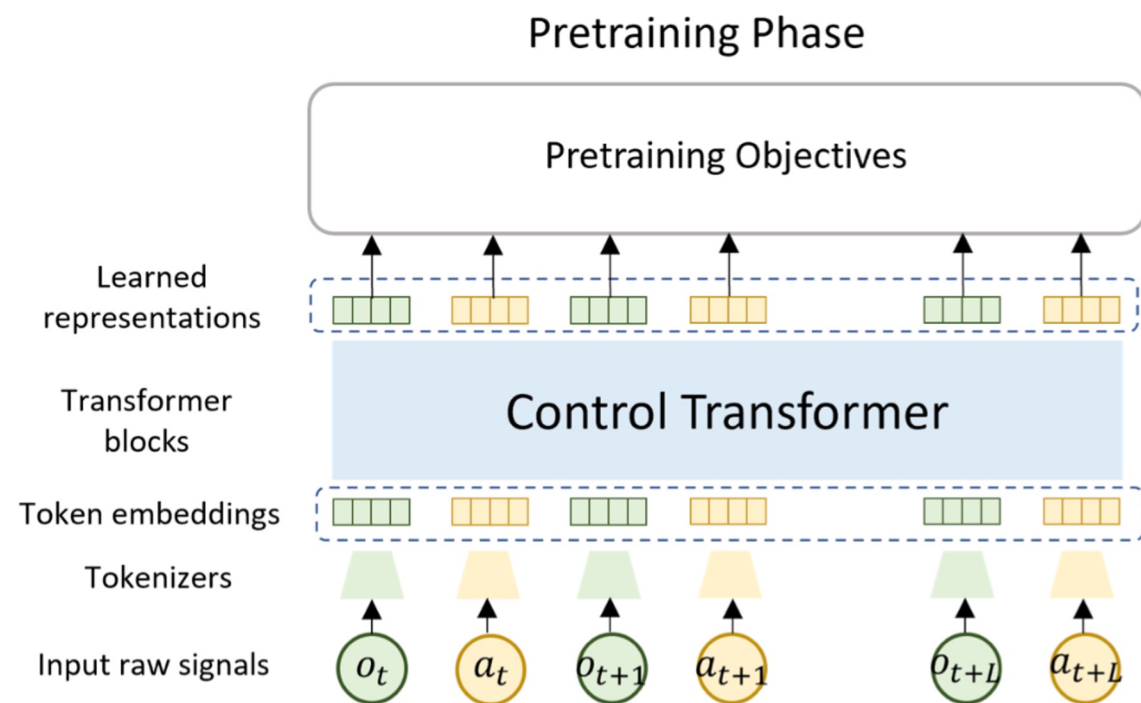
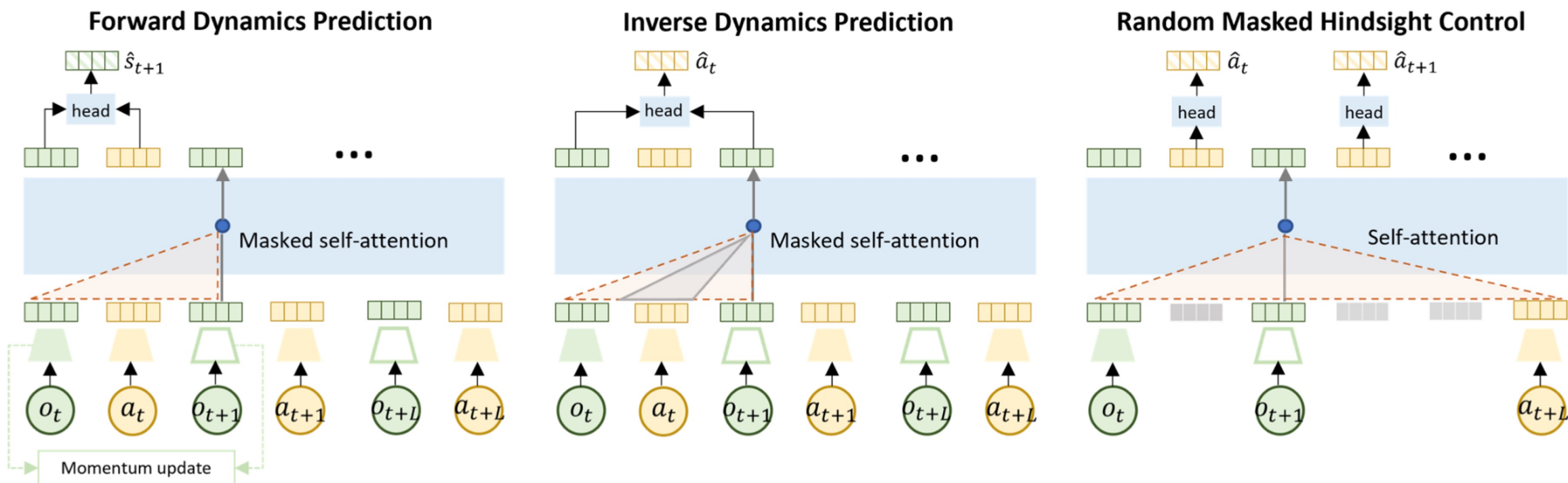


Figure 6 | Success rate (see main text) for R2D3 across all tasks with at least one successful seed, as a function of demo ratio. The square markers for each demo ratio denote the mean success rate, and the error bars show a bootstrapped estimate of the [25, 75] percentile interval for the mean estimate. The lower demo ratios consistently outperform the higher demo ratios across the suite of tasks.

SMART:



SMART:



06

Обучение по поведению в среде

VPT: Video PreTraining

Collect Internet data



Search the web

70K hours of
unlabeled video

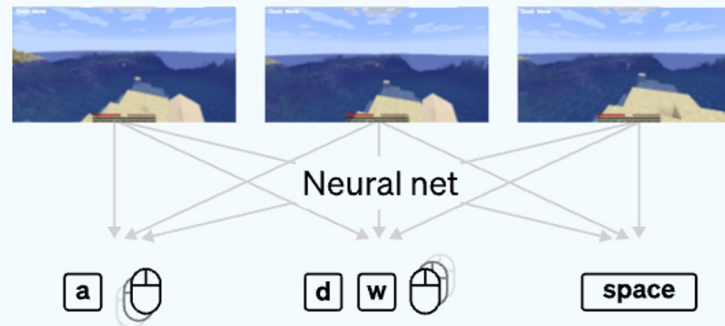
Train the Inverse Dynamics Model (IDM)



Contractors produce data

2K hours of video
*labeled with mouse
and keyboard actions*

Train a model to predict actions
given past and future frames



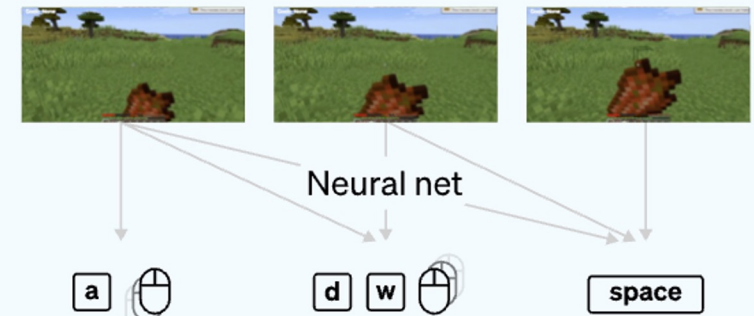
Train the VPT Foundation Model



Label videos with IDM

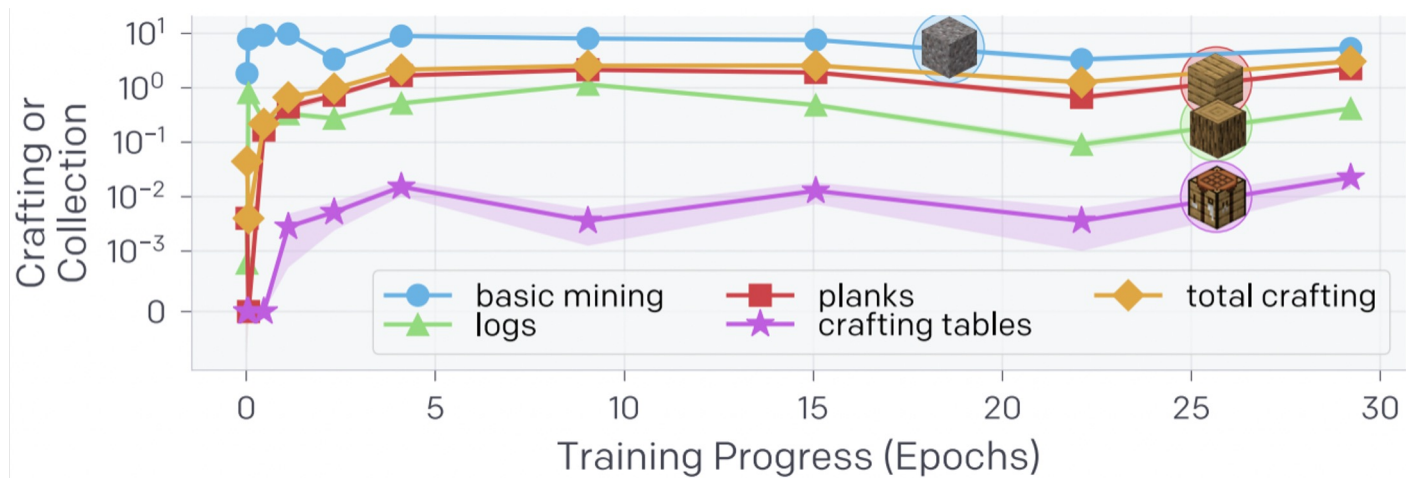
70K hours of video
*labeled with mouse
and keyboard actions*

Train a model to predict actions
given only past frames



VPT:

$$\min_{\theta} \sum_{t \in [1 \dots T]} -\log \pi_{\theta}(a_t | o_1, \dots, o_t), \text{ where } a_t \sim p_{\text{IDM}}(a_t | o_1, \dots, o_t, \dots, o_T)$$



~9 дней на 720 V100 GPUs

VPT:

Проблема: в обычных policy gradient методах стратегия и функция полезности обучаются вместе, но им нужны разные режимы обучения. Стратегию нельзя сильно менять, иначе обучение становится нестабильным, а функцию полезности хочется обучать дольше. PPG решает это, разделяя обучение на две фазы:

- ▶ Фаза стратегии агент обновляет стратегию действий, чтобы получать больше награды.
- ▶ Вспомогательная фаза: агент лучше обучает функцию ценности и представления, стараясь не сильно менять политику.

Algorithm 1 PPG

```
for phase = 1, 2, ... do
  Initialize empty buffer  $B$ 
  for iteration = 1, 2, ...,  $N_\pi$  do
    Perform rollouts under current policy  $\pi$ 
    Compute value function target  $\hat{V}_t^{\text{targ}}$  for each state  $s_t$ 
    for epoch = 1, 2, ...,  $E_\pi$  do
      Optimize  $L^{\text{clip}} + \beta_S S[\pi]$  wrt  $\theta_\pi$ 
    for epoch = 1, 2, ...,  $E_V$  do
      Optimize  $L^{\text{value}}$  wrt  $\theta_V$ 
    Add all  $(s_t, \hat{V}_t^{\text{targ}})$  to  $B$ 
  Compute and store current policy  $\pi_{\theta_{old}}(\cdot|s_t)$  for all states  $s_t$  in  $B$ 
  for epoch = 1, 2, ...,  $E_{aux}$  do
    Optimize  $L^{\text{joint}}$  wrt  $\theta_\pi$ , on all data in  $B$ 
    Optimize  $L^{\text{value}}$  wrt  $\theta_V$ , on all data in  $B$ 
```

for epoch = 1, 2, ..., E_π do
Optimize $L^{\text{clip}} + \beta_S S[\pi]$ wrt θ_π entropy bonus

$$L^{\text{clip}} = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$
$$L^{\text{value}} = \hat{\mathbb{E}}_t \left[\frac{1}{2} (V_{\theta_V}(s_t) - \hat{V}_t^{\text{targ}})^2 \right]$$
$$L^{\text{joint}} = L^{\text{aux}} + \beta_{\text{clone}} \cdot \hat{\mathbb{E}}_t [KL[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]]$$
$$L^{\text{aux}} = \frac{1}{2} \cdot \hat{\mathbb{E}}_t [(V_{\theta_\pi}(s_t) - \hat{V}_t^{\text{targ}})^2]$$

VPT:

При обучении важно не забыть прошлые полезные знания:

- ▶ Добавим KL на референсную модель обученную на прошлом этапе
- ▶ Будем постепенно ослаблять коэффициент при такой регуляризации

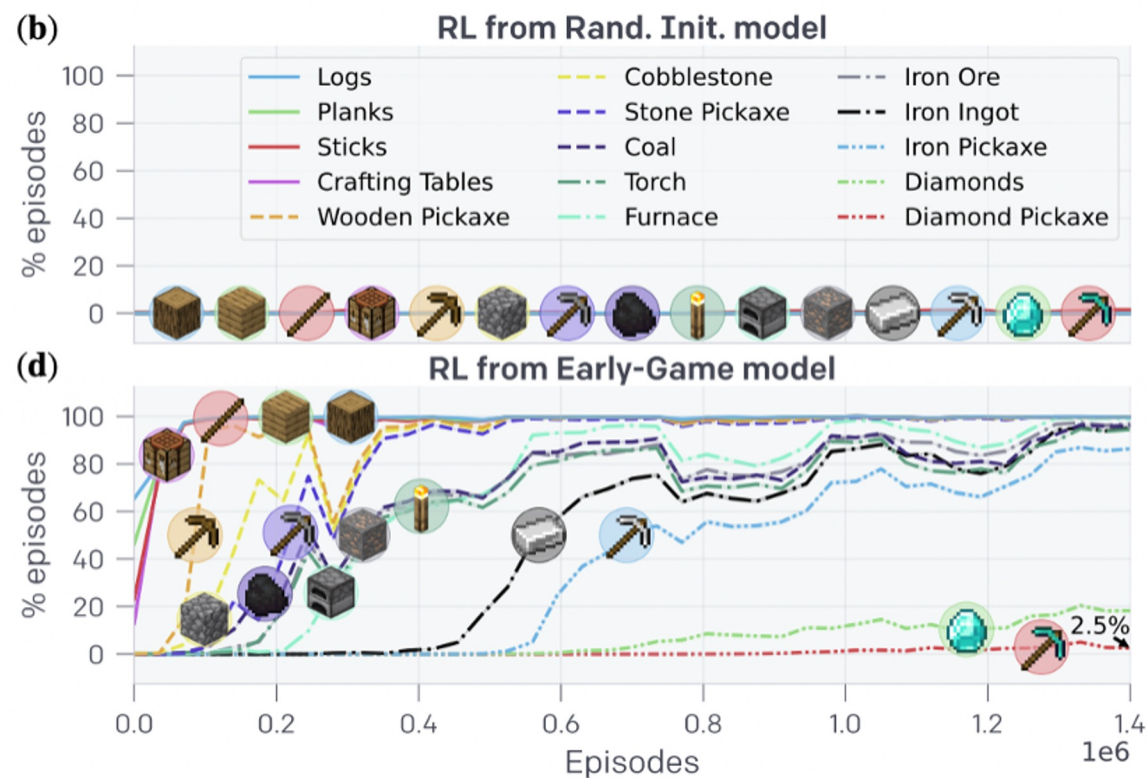
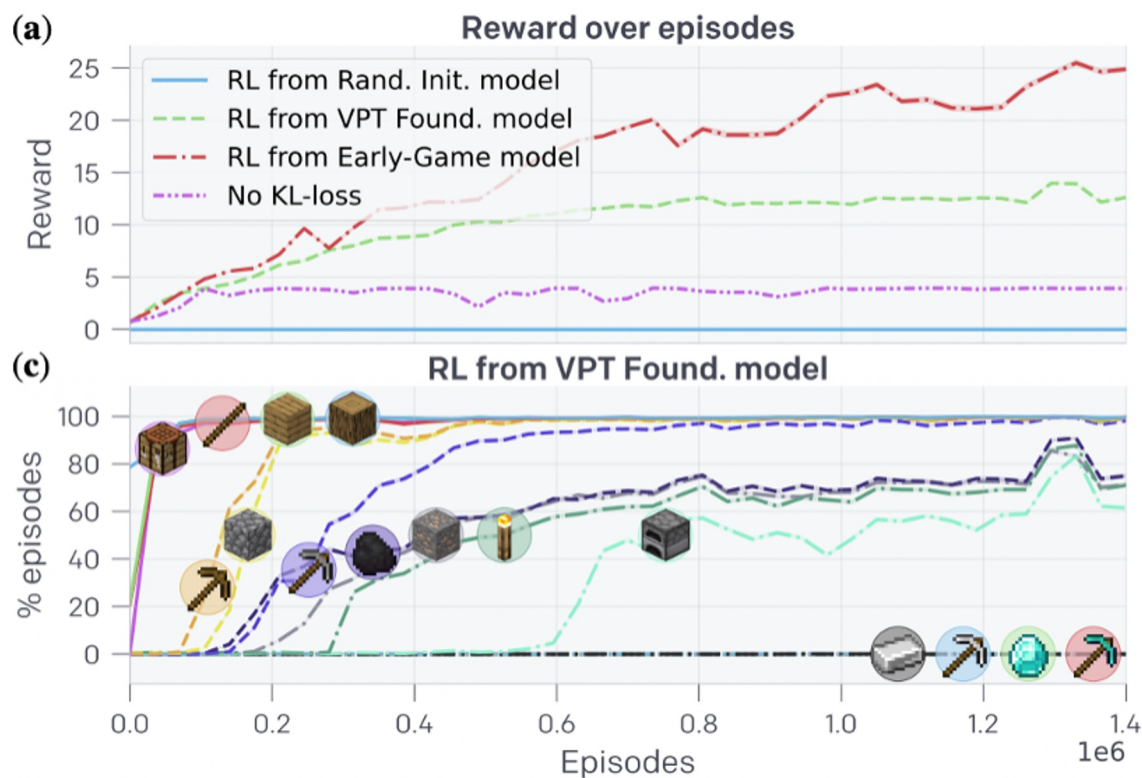
$$L_{klpt} = \rho \text{KL}(\pi_{pt}, \pi_{\theta})$$

entropy loss is replaced with the KL loss between the policy being trained and the pretrained one

Algorithm 1 PPG

```
for phase = 1, 2, ... do
  Initialize empty buffer  $B$ 
  for iteration = 1, 2, ...,  $N_{\pi}$  do                                ▷ Policy Phase
    Perform rollouts under current policy  $\pi$ 
    Compute value function target  $\hat{V}_t^{\text{targ}}$  for each state  $s_t$ 
    for epoch = 1, 2, ...,  $E_{\pi}$  do                                  ▷ Policy Epochs
      Optimize  $L^{\text{clip}} + \beta_S S[\pi]$  wrt  $\theta_{\pi}$ 
    for epoch = 1, 2, ...,  $E_V$  do                                  ▷ Value Epochs
      Optimize  $L^{\text{value}}$  wrt  $\theta_V$ 
    Add all  $(s_t, \hat{V}_t^{\text{targ}})$  to  $B$ 
  Compute and store current policy  $\pi_{\theta_{\text{old}}}(\cdot|s_t)$  for all states  $s_t$  in  $B$ 
  for epoch = 1, 2, ...,  $E_{\text{aux}}$  do                                ▷ Auxiliary Phase
    Optimize  $L^{\text{joint}}$  wrt  $\theta_{\pi}$ , on all data in  $B$ 
    Optimize  $L^{\text{value}}$  wrt  $\theta_V$ , on all data in  $B$ 
```

VPT:



VPT:



Figure 16: Items obtained when RL fine-tuning from the early-game model without a KL loss. The model learns to obtain all items that the early-game model can craft zero-shot, which are logs, planks, sticks, and a crafting table. In contrast to the treatment with a KL-penalty, it does not learn any items beyond these initial four, likely because skills that are not performed zero-shot, and for which the model thus does not initially see any reward, are catastrophically forgotten while the first four items are learned.

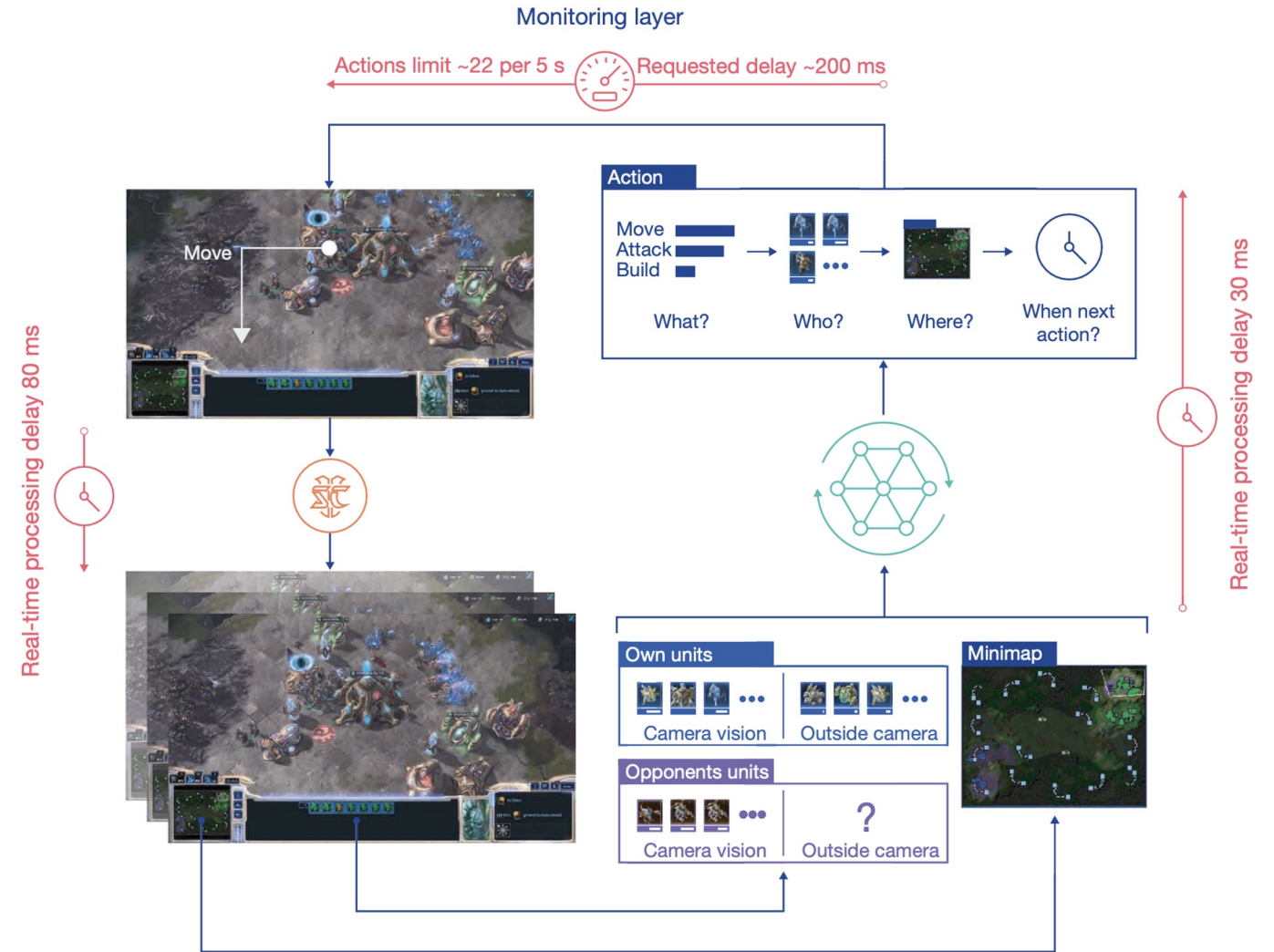
AlphaStar:

Наблюдение

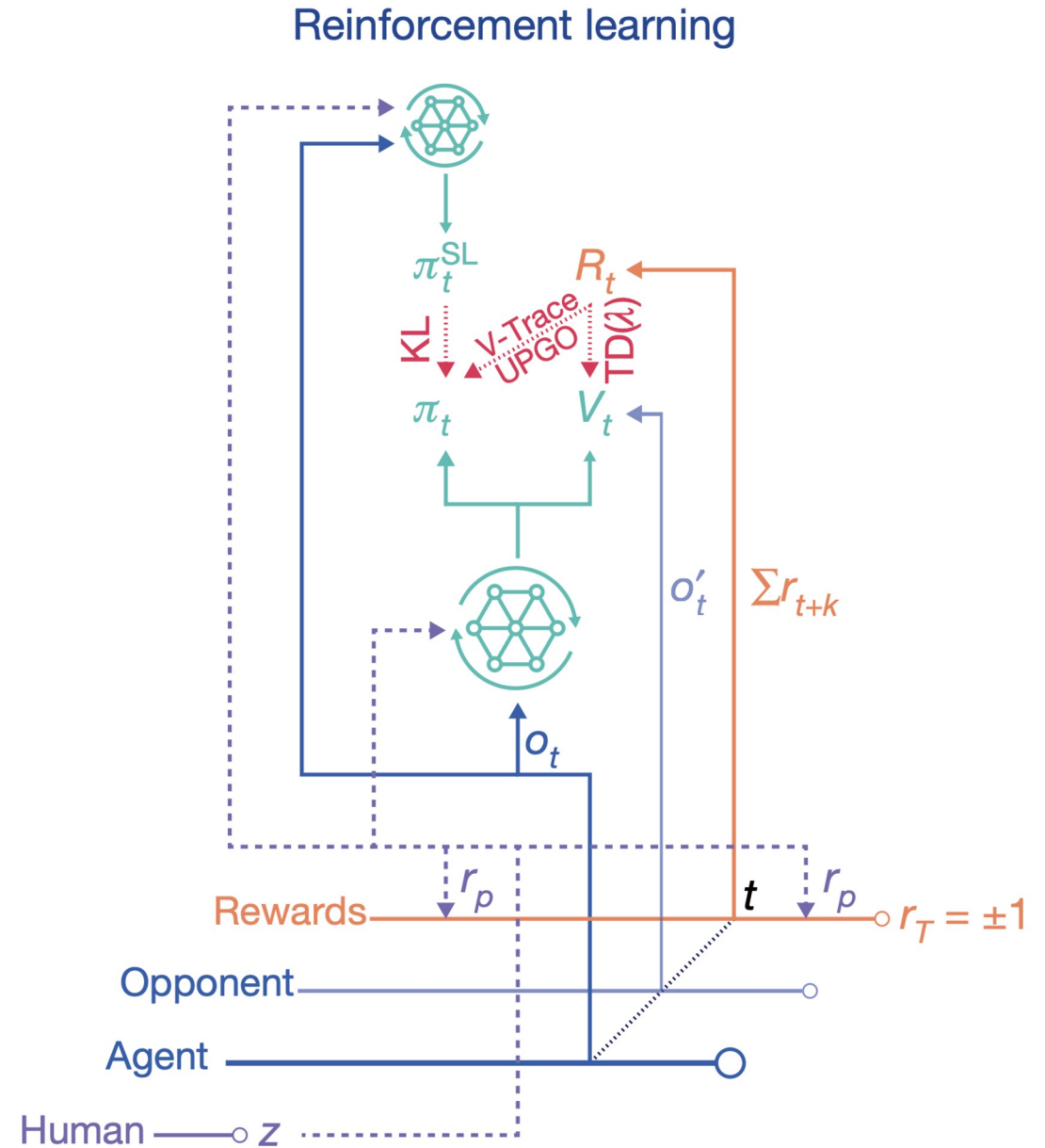
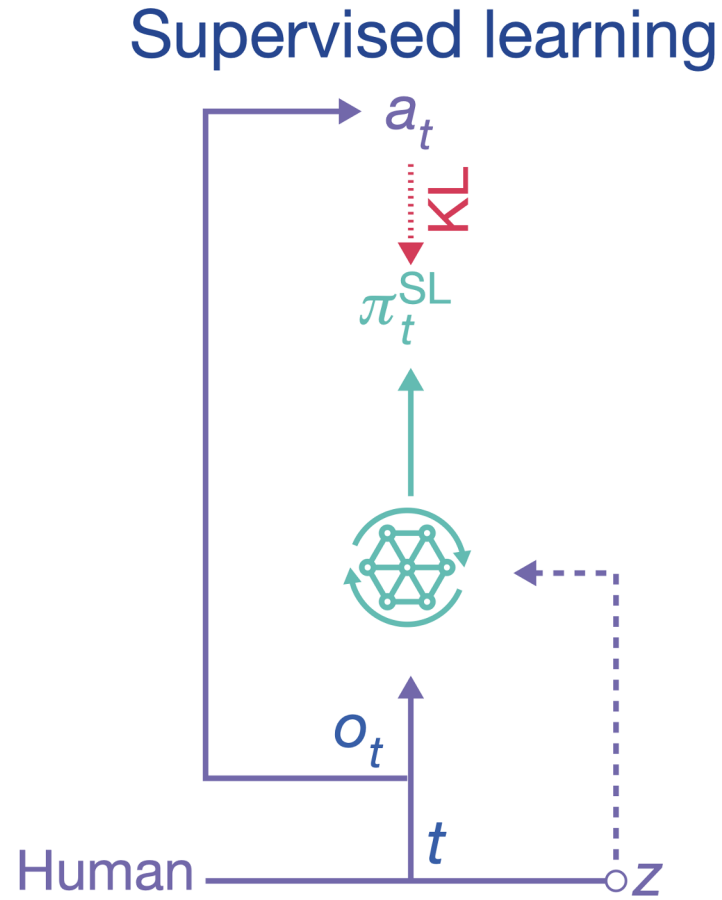
- ▶ Мини-карта
- ▶ Список юнитов

Действия

- ▶ Управление выбранной группой



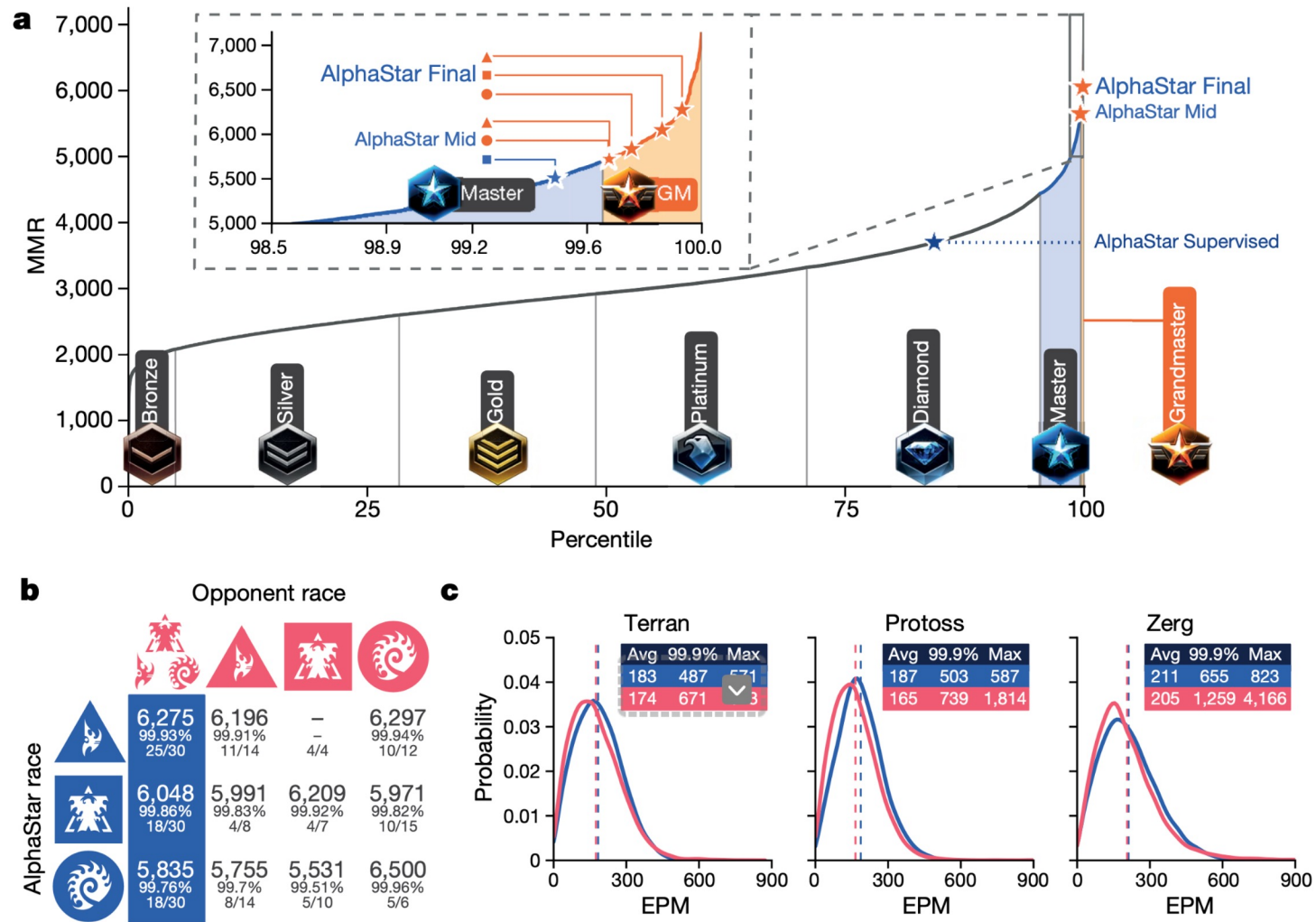
AlphaStar:



AlphaStar:

MMR: рейтинг силы игрока или агента. Он растёт после побед и падает после поражений, особенно если результат был неожиданным

- ▶ Много агентов играют друг против друга, а не против одного фиксированного соперника
- ▶ Старые версии агентов сохраняются в лиге, чтобы новые агенты не забывали старые стратегии
- ▶ MMR используется для оценки силы агентов и сравнения их уровня игры



Вопросы?